

Project Pack-a-mal có 2 thành phần chính:

- Dynamic Analysis Module
- Web APIs

1. Dynamic Analysis Module:

- Mục đích: phân tích các gói phần mềm từ nhiều hệ sinh thái open source bằng kỹ thuật phân tích động (dynamic analysis), tóm gọn là phân tích khi các gói phần mềm đang chạy
- Chức năng chính:
 - + **Runtime Behavior**: theo dõi quá trình cài đặt, import, và thực thi package trong môi trường sandbox.
 - + **System Call Tracking**: dùng trace để giám sát các thao tác, lệnh hệ thống, và tương tác hệ thống.
 - + **Network Activity Analysis**: ghi lại các truy vấn DNS và kết nối mạng
 - + **File Operation Monitoring**: giám sát quá trình đọc, ghi, xóa file trong quá trình chạy package.
 - + **Execution Logging**: Lưu lại các thông tin module đã chạy.
- Các modules:
 - + **Analysis image**: phân tích dynamic và static
 - + **Sandboxes**: môi trường container để chạy package an toàn
 - + **Schedule**
 - + **Worker**

2. Web APIs

- Mục đích: cung cấp giao diện và Rest API (giao thức truyền tin) để người dùng submit package, nhận report, quản lý các phân.
- Chức năng chính:
 - a. **Web Dashboard**
 - + **Package Discovery**: Tìm kiếm và duyệt các package từ nhiều hệ sinh thái.
 - + **Interactive Analysis**: Submit package để phân tích trực tiếp.
 - + **Real-time Status**: Theo dõi tiến trình phân tích và vị trí trong queue.
 - + **Report Visualization**: Xem báo cáo chi tiết với các phát hiện bảo mật.
 - b. **Analysis Capabilities**
 - + **Dynamic Analysis**: Giám sát hành vi runtime, hoạt động mạng, thao tác hệ thống, lệnh chạy, domain/IP tracking.
 - + **Static Analysis**:
 - + **Bandit4Mal**: Scan lỗ hổng bảo mật.
 - + **Malcontent**: Phát hiện nội dung độc hại.
 - + **LastPyMile**: Kiểm tra sự khác biệt source package.
 - + **Typosquatting Detection**: Phát hiện package có tên gần giống các package nổi tiếng.
 - + **Source Code Finder**: Tìm repo source code (PyPI, npm).
 - c. **REST APIs**
 - + Truy cập tất cả tính năng phân tích bằng API.

- + Xác thực API key, giới hạn tốc độ.
- + Quản lý queue và trạng thái task.
- + Hỗ trợ định dạng **Package URL (PURL)**.

Tiến hành chạy local + dùng Docker

- make build

☐	Name	Tag	Image ID	Created	Size	Actions
☐	buiminh23/static-anal	latest	4e40b6dee6e1	44 minutes ag	280.9 MB	
☐	buiminh23/analysis	latest	e85e80a22c53	43 minutes ag	983.12 MB	
☐	buiminh23/scheduler	latest	5a4a5a95260d	41 minutes ag	73.15 MB	
☐	buiminh23/dynamic-a	latest	ceb5b5530513	46 minutes ag	3.27 GB	

- make push
- make sync
- make test_dynamic_analysis:
result:

```

Finished analysis

Ecosystem:          rubygems
Package:            guwor_palindrome
Version:
Location:           remote
Dynamic analysis results: /tmp/results
Static analysis results: /tmp/staticResults
File write results:  /tmp/writeResults
Analyzed package saved: /tmp/analyzedPackages
Debug logs:         /tmp/dockertmp
Strace logs:        /tmp/straceLogs
-----
Dynamic analysis test passed

```

- make test_static_analysis:
result:

Finished analysis

Ecosystem: npm
Package: async
Version:
Location: remote
Dynamic analysis results: /tmp/results
Static analysis results: /tmp/staticResults
File write results: /tmp/writeResults
Analyzed package saved: /tmp/analyzedPackages
Debug logs: /tmp/dockertmp
Strace logs: /tmp/straceLogs

Static analysis test passed

- Output folder:

 analyzedPackages	1/16/2026 9:48 PM	File folder
 dockertmp	1/16/2026 9:48 PM	File folder
 results	1/16/2026 9:48 PM	File folder
 staticResults	1/16/2026 9:37 PM	File folder
 straceLogs	1/16/2026 9:37 PM	File folder
 writeResults	1/16/2026 9:48 PM	File folder

- Kết quả nhận được:

📁 symfony	1/16/2026 9:48 PM	File folder	
📄 async.json	1/16/2026 9:46 PM	JSON File	86 KB
📄 async.json.attrs	1/16/2026 9:46 PM	ATTRS File	1 KB
📄 guwor_palindrome.json	1/16/2026 9:46 PM	JSON File	86 KB
📄 guwor_palindrome.json.attrs	1/16/2026 9:46 PM	ATTRS File	1 KB
📄 itoa.json	1/16/2026 9:46 PM	JSON File	86 KB
📄 itoa.json.attrs	1/16/2026 9:46 PM	ATTRS File	1 KB
📄 requests.json	1/16/2026 9:46 PM	JSON File	86 KB
📄 requests.json.attrs	1/16/2026 9:46 PM	ATTRS File	1 KB

Đề xuất ý tưởng

- hiện tại Minh và Nhân không có ý tưởng nào hay để đề xuất cho việc mô phỏng các dịch vụ Internet.

Hiểu được các thành phần của project này: dockers.

Có 4 Dockers chính:

- Analysis
 - + Phân tích package trong môi trường cách ly, chạy sandbox với adVisor để phát hiện malware, capture network traffic và system calls.
- Scheduler
 - + Lập lịch và quản lý hàng đợi các task phân tích package, phân phối công việc cho các worker.
- Dynamic Analysis Sandbox
 - + Thực thi code của package thực tế để quan sát hành vi: network calls, file operations, process spawning. Hỗ trợ 6 ngôn ngữ: Python, Node.js, Ruby, Rust, PHP, Java.
- Static Analysis Sandbox
 - + Phân tích code mà không chạy: quét dependencies kiểm tra file structure, phát hiện pattern nguy hiểm trong source code.

Demo thực hiện 2 Docker giao tiếp với nhau.

- Cấu trúc:

```

ProjectTest/
├── api/
│   ├── app.py           # Code server Flask (có API /chat)
│   ├── Dockerfile       # Cài python, iputils-ping, netcat-openbsd
│   └── requirements.txt  # Thư viện: flask
├── flarenet/
│   └── Dockerfile       # Cài ubuntu, curl, iputils-ping, netcat-openbsd
├── docker-compose.yml   # Cấu hình 2 service và network
└── README.md            # File hướng dẫn này

```

- Mục tiêu:
- + Service 1(API): Đóng vai trò Server, chạy ứng dụng Python Flask.
- + Service 2(Flarenet): Đóng vai trò Client, chạy hệ điều hành Ubuntu để thực hiện các truy vấn mạng.
- + Mục đích là chứng minh khả năng giao tiếp giữa các container.

docker-compose.yml

```

1  version: '3.8'
2
3  services:
4    api:
5      build: ./api
6      container_name: project_api
7      ports:
8        - "5000:5000"
9      networks:
10       - app_network
11      restart: always
12
13     flarenet:
14       build: ./flarenet
15       container_name: project_flarenet
16       networks:
17         - app_network
18       depends_on:
19         - api
20
21  networks:
22    app_network:
23      driver: bridge

```

Flarenet-Dockerfile

```
flarenet > Dockerfile
1 FROM ubuntu:22.04
2
3 RUN apt-get update && apt-get install -y curl iputils-ping netcat-openbsd
4
5 CMD ["sleep", "infinity"]
```

Api app.py

```
api > app.py > ...
1 from flask import Flask, request, jsonify
2 import socket
3
4 app = Flask(__name__)
5
6 @app.route('/')
7 def home():
8     return jsonify({"message": "API Ready", "hostname": socket.gethostname()})
9
10 @app.route('/chat', methods=['POST'])
11 def chat():
12     # 1. Nhận tin nhắn từ Flarenet gửi sang
13     data = request.json
14     message_received = data.get('message', '')
15     sender = data.get('sender', 'Unknown')
16
17     print(f"Update: Received '{message_received}' from {sender}", flush=True)
18
19     # 2. Gửi lời chào ngược lại (Response)
20     response_message = f"Hello {sender}, I am API Server. I received your message: '{message_received}'"
21
22     return jsonify({
23         "status": "success",
24         "reply": response_message
25     })
26 # -----
27
28 if __name__ == '__main__':
29     app.run(host='0.0.0.0', port=5000)
```

Api Dockerfile

```
api > Dockerfile
1 FROM python:3.9-slim
2
3 WORKDIR /app
4
5 RUN apt-get update && apt-get install -y iputils-ping netcat-openbsd
6
7 COPY requirements.txt .
8 RUN pip install --no-cache-dir -r requirements.txt
9
10 COPY . .
11
12 EXPOSE 5000
13
14 CMD ["python", "app.py"]
```

Kết quả sau khi chạy và giao tiếp giữa 2 docker:

Có nhiều cách để test giữa 2 docker có thể giao tiếp với nhau không.

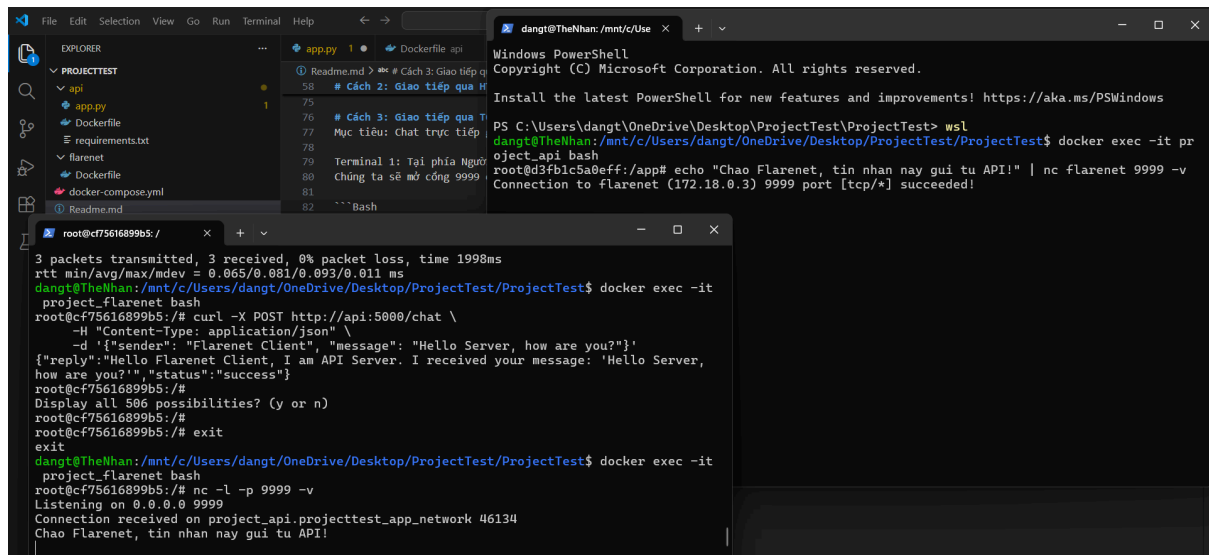
Cách 1: Sử dụng giao thức **ICMP** để kiểm tra tính sẵn sàng (Availability) và độ trễ của kết nối giữa hai node mạng.

```
dangt@TheNhan:/mnt/c/Users/dangt/OneDrive/Desktop/ProjectTest/ProjectTest$ docker exec -it project_api ping flarenet -c 4
PING flarenet (172.18.0.3) 56(84) bytes of data.
64 bytes from project_flarenet.projecttest_app_network (172.18.0.3): icmp_seq=1 ttl=64 time=0.085 ms
64 bytes from project_flarenet.projecttest_app_network (172.18.0.3): icmp_seq=2 ttl=64 time=0.065 ms
64 bytes from project_flarenet.projecttest_app_network (172.18.0.3): icmp_seq=3 ttl=64 time=0.093 ms
```

Cách 2: Sử dụng giao thức **HTTP** để gửi request giả lập người dùng. Đây là công cụ chuẩn để test REST API.

```
dangt@TheNhan:/mnt/c/Users/dangt/OneDrive/Desktop/ProjectTest/ProjectTest$ docker exec -it project_flarenet bash
root@cf75616899b5:/# curl -X POST http://api:5000/chat \
-H "Content-Type: application/json" \
-d '{"sender": "Flarenet Client", "message": "Hello Server, how are you?"}'
{"reply": "Hello Flarenet Client, I am API Server. I received your message: 'Hello Server, how are you?',"status": "success"}
```

Cách 3: Sử dụng 2 terminal, terminal đầu chạy lệnh để vào trạng thái listening chờ đợi kết nối và thông tin ping qua từ terminal thứ 2.



```
File Edit Selection View Go Run Terminal Help
EXPLORER
PROJECTTEST
  api
    app.py
    Dockerfile
    requirements.txt
  flarenet
    Dockerfile
  docker-compose.yml
  README.md

Dockerfile api
1  # Cách 2: Giao tiếp qua HTTP
2  # Cách 3: Giao tiếp qua TCP
3  # Mục tiêu: Chat trực tiếp
4  # Terminal 1: Tại phía Người dùng
5  # Chúng ta sẽ mở cổng 9999
6  # Bash

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.
Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows
PS C:\Users\dangt\OneDrive\Desktop\ProjectTest\ProjectTest> wsl
dangt@TheNhan:/mnt/c/Users/dangt/OneDrive/Desktop/ProjectTest/ProjectTest$ docker exec -it pr
ject_api bash
root@d3f1c3a0eff:/app# echo "Chao Flarenet, tin nhan nay gui tu API!" | nc flarenet 9999 -v
Connection to flarenet (172.18.0.3) 9999 port [tcp/*] succeeded!

root@cf75616899b5:/#
3 packets transmitted, 3 received, 0% packet loss, time 1998ms
rtt min/avg/max/mdev = 0.065/0.081/0.093/0.011 ms
dangt@TheNhan:/mnt/c/Users/dangt/OneDrive/Desktop/ProjectTest/ProjectTest$ docker exec -it
project_flarenet bash
root@cf75616899b5:/# curl -X POST http://api:5000/chat \
-H "Content-Type: application/json" \
-d '{"sender": "Flarenet Client", "message": "Hello Server, how are you?"}'
{"reply": "Hello Flarenet Client, I am API Server. I received your message: 'Hello Server,
how are you?',"status": "success"}
root@cf75616899b5:/#
Display all 506 possibilities? (y or n)
root@cf75616899b5:/#
root@cf75616899b5:/# exit
exit
dangt@TheNhan:/mnt/c/Users/dangt/OneDrive/Desktop/ProjectTest/ProjectTest$ docker exec -it
project_flarenet bash
root@cf75616899b5:/# nc -l -p 9999 -v
Listening on 0.0.0.0 9999
Connection received on project_api.projecttest_app_network 46134
Chao Flarenet, tin nhan nay gui tu API!
```

KẾT QUẢ THỰC NGHIỆM

Qua quá trình thực hành, nhóm đã thực hiện thành công 3 kịch bản giao tiếp:

1. **Ping (Lớp Network):** **api** đã ping thấy **flarenet**, chứng minh lớp mạng Bridge hoạt động ổn định và DNS phân giải đúng IP.
2. **HTTP Request (Lớp Application):** **flarenet** gửi JSON sang **api** và nhận được phản hồi **200 OK**. Chứng minh ứng dụng Web hoạt động đúng trên môi trường container.
3. **TCP Socket (Lớp Transport):** Sử dụng **Netcat** để chat trực tiếp giữa 2 terminal. Chứng minh các cổng (Port) được mở thông suốt trong mạng nội bộ.

Service Simulation module: simulate network services, such as HTTP, DNS.

- Cấu trúc:

```
service-simulation-module/  
├── docker-compose.yml  
├── inetsim/  
│   ├── Dockerfile  
│   └── entrypoint.sh  
├── service-simulation/  
│   ├── Dockerfile  
│   └── app/  
│       ├── main.py  
│       ├── api/  
│       │   └── server.py  
│       ├── collector/  
│       │   └── logs.py  
│       └── config/  
│           └── inetsim.py  
└── shared/  
    ├── config/  
    │   └── etc/inetsim/  
    │       └── inetsim.conf  
    └── logs/
```

docker-compose.yml


```

> Run All Services
services:
  > Run Service
  inetsim:
    build: ./inetsim
    container_name: inetsim
    ports:
      - "53:53/udp" #cổng DNS UDP
      - "8080:80" #cổng HTTP
    volumes:
      - ./shared/config/etc/inetsim:/etc/inetsim #thư mục cấu hình inetsim
      - ./shared/logs:/logs #thư mục chung lưu log
      - ./shared/logs/inetsim:/var/log/inetsim #thư mục lưu log của inetsim
    networks:
      simulation_network:
        ipv4_address: 172.20.0.2 #static IP cho DNS resolution

  > Run Service
  service-simulation:
    build: ./service-simulation
    container_name: service-simulation
    depends_on:
      - inetsim #đảm bảo inetsim khởi động trước
    ports:
      - "5000:5000" #cổng API Flask
    volumes:
      - ./shared/logs:/logs #thư mục chung lưu log
      - ./shared/config:/config #thư mục cấu hình chung
      - ./shared/config/etc/inetsim:/etc/inetsim #thư mục cấu hình inetsim
    dns:
      - 172.20.0.2 #IP của inetsim container (DNS nameserver)
    networks:
      - simulation_network #mạng chung cho các dịch vụ

networks:
  simulation_network:
    driver: bridge #sử dụng mạng cầu nối để các container có thể giao tiếp với nhau qua tên dịch vụ
    ipam:
      config:
        - subnet: 172.20.0.0/24 #subnet cho static IP assignment

```

1. Đây là file docker compose để mô phỏng service network, các thành phần:

- Container inetsim:
 - + Chức năng: máy chủ mô phỏng các dịch vụ mạng
 - + Ports:
 - + 53:53/udp - DNS server
 - + 8080/80 - HTTP server
 - + Volumes:
 - + Cấu hình INetSim từ shared\config\etc\inetsim
 - + log files trong shared\logs
 - + static IP: 172.20.0.2 (dùng làm DNS nameserver)

2. Container service simulation:

- a. Chức năng: API Flask để quản lý và theo dõi hệ thống
- b. Port 5000:5000 - Flask API
- c. Phụ thuộc: bắt buộc khởi động sau container INetSim
- d. Volumes: chia sẻ logs và cấu hình cho INetSim

3. Network Configuration

- a. Tên mạng: simulation_network

- b. Driver: Bridge
- c. subnet: 172.20.0.0/24
- d. Mục đích: cho phép các container giao tiếp với nhau

INetsim

- Dockerfile:

```
FROM ubuntu:22.04

RUN apt-get update && apt-get install -y \
    inetsim \
    net-tools \
    curl \
    dnsmasq

COPY entrypoint.sh /entrypoint.sh
#thêm quyền execute cho file entrypoint.sh, chmod (change mod) r,w,x
RUN chmod +x /entrypoint.sh
#khi khởi động container sẽ chạy entrypoint.sh
CMD ["/entrypoint.sh"]
```

1. Base image:
 - a. FROM ubuntu:22.04: sử dụng ubuntu làm hệ điều hành nền (nếu sử dụng phiên bản cao hơn thì INetsim sẽ không chạy được toàn bộ tính năng do INetsim không còn cập nhật thêm từ 2021 trở đi).
2. Cài đặt các gói phần:
 - a. cài các service của INetsim
3. Cấu hình Entrypoint:
 - a. chức năng: thực hiện các bước chuẩn bị trước khi khởi động container (nội dung nằm ở bên dưới)

- Entrypoint.sh:

```
#!/bin/bash

mkdir -p /var/log/inetsim #tạo thư mục log nếu chưa tồn tại
chown -R inetsim:inetsim /var/log/inetsim #thay đổi chủ sở hữu của thư mục log thành user inetsim
chmod -R 775 /var/log/inetsim #thay đổi quyền truy cập của thư mục log, 7 cho chủ sở hữu (r,w,x), 7 cho nhóm

touch /var/log/inetsim/main.log #tạo file log chính nếu chưa tồn tại
chown inetsim:inetsim /var/log/inetsim/main.log #thay đổi chủ sở hữu của file log thành user inetsim
chmod 664 /var/log/inetsim/main.log #thay đổi quyền truy cập của file log, 6 cho chủ sở hữu (r,w), 6 cho nhóm

exec /usr/bin/inetsim --data /var/lib/inetsim --conf /etc/inetsim/inetsim.conf --log-dir /var/log/inetsim #chạy inetsim
```

1. Nội dung:

- a. Tạo thư mục log nếu chưa tồn tại.
- b. Thay đổi chủ sở hữu của thư mục log thành user inetsim.
- c. thay đổi quyền truy cập của thư mục log.
- d. tạo file log chính nếu chưa tồn tại.
- e. thay đổi chủ sở hữu file log.
- f. thay đổi quyền truy cập của file log.

Service-Simulation

- Dockerfile

```
FROM python
COPY app /app
WORKDIR /app

RUN pip install flask

CMD python main.py
```

- main.py

```
from config.inetsim import generate_config
from api.server import app
import time, socket #import module time để tạm dừng, socket để kiểm tra kết nối mạng

generate_config()

while True:
    try:
        socket.create_connection(("inetsim", 80), timeout=5) #kiểm tra kết nối tới dịch vụ inetsim trên c
        break
    except OSError: #nếu không kết nối được thì tạm dừng 1 giây và thử lại
        time.sleep(1)

# Start Flask API server
if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000) #chạy server Flask trên tất cả các địa chỉ mạng với cổng 5000
```

1. Nội dung:

- a. import module time để tạm dừng, socket để kiểm tra kết nối mạng
- b. kiểm tra kết nối tới dịch vụ inetsim trên port 80, nếu không kết nối được thì tạm dừng 1 giây và thử lại
- c. chạy server Flask API trên tất cả các địa chỉ mạng với cổng 5000 (hiện tại tính năng API này chưa được sử dụng tới (**đang trong quá trình học và thử nghiệm server API**))

- server.py

```
from flask import Flask

app = Flask(__name__) #khởi tạo ứng dụng Flask

@app.route("/status") #định nghĩa route /status
def status():
    return {"service": "simulation", "status": "running"} #trả về trạng thái dịch vụ dưới dạng JSON
```

1. Hiện tại đang trong quá trình học và thử nghiệm , nên tính năng này chưa được áp dụng ở thời điểm hiện tại.

- logs.py

```
def collector():
    with open('/logs/inetsim.log') as log_file:
        for line in log_file:
            if "DNS" in line or "HTTP" in line:
                print("Event: ", line.strip())
```

1. Đang thử nghiệm, tính năng thu thập các log từ containers về , để nhận biết rằng các log có được thu thập hay bị lỗi trong quá trình thu thập.

- inetsim.py

```

import os #import module os để thao tác với hệ thống file

def generate_config():
    config = """service_bind_address 0.0.0.0
dns_default_ip 127.0.0.1

start_service http
start_service dns

http_default_fakefile    sample.html text/html

dns_bind_port 53
"""

    config_dir = "/etc/inetsim"
    config_file_path = os.path.join(config_dir, "inetsim.conf") #đường dẫn tới f

    if os.path.exists(config_file_path): #nếu file cấu hình đã tồn tại thì xóa đ
        os.remove(config_file_path)

    os.makedirs(config_dir, exist_ok=True) #tạo thư mục cấu hình nếu chưa tồn tạ
    with open(config_file_path, "w") as config_file:
        config_file.write(config)

```

1. Nội dung:
 - a. import module os của python để thao tác với hệ thống file
 - b. ý tưởng là sau mỗi lần mình build lại 1 docker compose thì nó sẽ tự động generate ra file inetsim.conf, để người dùng (**không rành về công nghệ**) có thể tối ưu hóa quá trình chạy.
2. Chức năng:
 - a. Nếu file cấu hình đã tồn tại thì xóa file đó đi, rồi tạo 1 thư mục cấu hình mới, exist_ok=True (**để khỏi báo lỗi khi không có file**), sau đó mở cấu hình file để ghi nội dung vào.

Quá trình hiện tại đã làm được:

1. Docker
 - a. Docker compose với 2 containers.
 - b. Network configuration với subnet riêng (172.20.0.0/24).
 - c. Volume mapping để chia sẻ logs và config
2. Server-Simulate Container
 - a. Dockerfile python

- b. Config generate **service-simulation\app\config\inetsim.py** - tạo file cấu hình INetSim
- 3. INetsim Container
 - a. Dockerfile ubuntu 22.04 + INetSim
 - b. Entrypoint scripts để setup permissions và khởi động
 - c. Cấu hình DNS (port 53) và HTTP(port 80).
 - d. static IP (172.20.0.2) làm DNS server.

Thực hiện chạy :

- Bước 1: Ngay tại folder của project này, chạy **powershell**:
- Bước 2: ghi Docker compose build
- Bước 3: ghi Docker compose up
 - + Output:

```

Attaching to insetsim, service-simulation
inetsim | INetSim 1.3.2 (2020-05-19) by Matthias Eckert & Thomas Hungenberg
inetsim | Using log directory: /var/log/inetsim/
inetsim | Using data directory: /var/lib/inetsim/
inetsim | Using report directory: /var/log/inetsim/report/
inetsim | Using configuration file: /etc/inetsim/inetsim.conf
inetsim | Parsing configuration file.
inetsim | Configuration file parsed successfully.
inetsim | === INetSim main process started (PID 1) ===
inetsim | Session ID: 1
inetsim | Listening on: 0.0.0.0
inetsim | Real Date/Time: 2026-01-20 11:43:25
inetsim | Fake Date/Time: 2026-01-20 11:43:25 (Delta: 0 seconds)
inetsim | Forking services...

service-simulation | * Serving Flask app 'api.server'
inetsim | * http_80_tcp - started (PID 14)
service-simulation | * Debug mode: off
service-simulation | WARNING: This is a development server. Do not use it in a production deployment. Use a production
WSGI server instead.
service-simulation | * Running on all addresses (0.0.0.0)
service-simulation | * Running on http://127.0.0.1:5000
service-simulation | * Running on http://172.20.0.3:5000
service-simulation | Press CTRL+C to quit

```

- Bước 4: mở powershell khác, ở Localhost (ổ C) hay trong chính folder đó cũng được.
- Bước 5: Test HTTP simulate : **curl http://localhost:8080 -UseBasicParsing**
 - + Output:

```

StatusCode      : 200
StatusDescription : OK
Content         : <html>
                  <head>
                    <title>INetSim default HTML page</title>
                  </head>
                  <body>
                    <p></p>
                    <p align="center">This is the default HTML page for INetSim HTTP server fake mode.</p>
                    <p align="cen...
RawContent      : HTTP/1.1 200 OK
                  Connection: Close
                  Content-Length: 258
                  Content-Type: text/html
                  Date: Tue, 20 Jan 2026 11:45:24 GMT
                  Server: INetSim HTTP Server

                  <html>
                  <head>
                    <title>INetSim default HTML pa...
Forms           : 
Headers         : {[Connection, Close], [Content-Length, 258], [Content-Type, text/html], [Date, Tue, 20 Jan 2026
                  11:45:24 GMT]...}
Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      : 
RawContentLength : 258

```

- Bước 6: Test DNS simulate: bạn có thể dùng **nslookup google.com 127.0.0.1**
- + Output:

```

Server:  www.inetsim.org
Address:  127.0.0.1

Name:     google.com.lan
Address:  127.0.0.1

```

So sánh 2 công cụ Inetsim và flare

- Số lượng services hỗ trợ
- Chất lượng các hồi đáp
- Ý tưởng so sánh khác?

Tích hợp Network Simulation vào Pack-A-Mal project

1. Tạo một package mẫu có kết nối tới một URL (không còn alive)
2. Thay đổi/Thêm code của project Pack-A-Mal để kiểm tra xem URL có alive hay không, nếu không alive thì điều hướng tới dịch vụ Inetsim/flare.

Tổng quan về INetSim và FakeNet-NG:

Trong phân tích mã độc động (Dynamic Analysis), việc ngăn chặn mã độc kết nối ra Internet thực tế nhưng vẫn phải đánh lừa mã độc rằng kết nối đã thành công là vô cùng quan trọng. Hai công cụ phổ biến nhất để thực hiện việc này là INetSim và FakeNet-NG.

Các thông tin quan trọng và so sánh công cụ INetSim và FakeNet-NG

INetSim:

- **Ưu điểm:** Hỗ trợ số lượng dịch vụ **vượt trội và đa dạng hơn**. INetSim được thiết kế để giả lập một hệ sinh thái mạng hoàn chỉnh.
- **Các dịch vụ nổi bật:** Ngoài các dịch vụ phổ biến (HTTP, HTTPS, DNS, FTP), INetSim còn hỗ trợ rất nhiều giao thức cũ hoặc ít phổ biến mà các mã độc phức tạp hoặc mã độc cũ vẫn dùng như: IRC (điều khiển Botnet), TFTP, NTP, Finger, Syslog, SMTP/POP3/IMAP (gửi nhận mail rác), Time, Daytime, Echo, v.v.
- **Đánh giá:** Rất mạnh mẽ cho việc phân tích các mã độc đa năng hoặc mã độc sử dụng các giao thức không phải HTTP.

FakeNet-NG (FLARE):

- **Đặc điểm:** Tập trung chủ yếu vào các giao thức **hiện đại và phổ biến nhất** mà mã độc ngày nay hay sử dụng.
- **Các dịch vụ nổi bật:** HTTP, HTTPS, SSL, DNS, FTP, SMTP, TCP/UDP listener cơ bản.
- **Đánh giá:** Số lượng services ít hơn INetSim, nhưng đủ dùng cho khoảng 80-90% các mẫu mã độc hiện đại (thường chỉ dùng HTTP/S và DNS).

Chất lượng và Tính linh hoạt của Hồi đáp (Response Quality)

INetSim:

- **Cơ chế:** Phản hồi dựa trên cấu hình tĩnh (Static Configuration). Ví dụ: Khi mã độc tải một file, INetSim sẽ trả về đúng file mặc định mà người dùng đã cấu hình (ví dụ: `sample.exe` hoặc `default.html`).
- **Chất lượng:** Phản hồi chuẩn xác theo RFC nhưng khá "cứng nhắc". Nếu mã độc kiểm tra nội dung file trả về (ví dụ: mã độc yêu cầu file `.jpg` nhưng INetSim cấu hình mặc định trả về file `.html`), mã độc có thể phát hiện ra nó đang bị giả lập.
- **Khả năng tùy biến:** Cần chỉnh sửa file cấu hình (`inetsim.conf`) thủ công để thay đổi nội dung trả về.

FakeNet-NG (FLARE):

- **Cơ chế:** Phản hồi động theo ngữ cảnh (Dynamic/Context-aware Response). Đây là điểm mạnh nhất của FakeNet-NG.
- **Chất lượng:** Thông minh hơn. Công cụ có khả năng tự động nhận diện loại file mà mã độc đang cố tải về (dựa trên phần mở rộng hoặc header) để tạo ra phản hồi phù hợp.
 - *Ví dụ:* Nếu mã độc yêu cầu `malware.exe`, FakeNet-NG sẽ trả về một file có cấu trúc PE (Portable Executable) hợp lệ. Nếu yêu cầu `image.png`, nó trả về file có header là PNG.
- **Tác dụng:** Giúp đánh lừa các mã độc có cơ chế kiểm tra tính toàn vẹn của dữ liệu tải về tốt hơn (Anti-sandbox).

Khả năng xử lý SSL/HTTPS

- **INetSim:** Sử dụng cơ chế Man-in-the-Middle với chứng chỉ số (Certificate) tự ký (Self-signed). Cần phải cài Root CA của INetSim vào máy nạn nhân để tránh lỗi SSL.
- **FakeNet-NG:** Cũng hỗ trợ giải mã SSL nhưng tích hợp sâu hơn vào Windows, việc xử lý các luồng dữ liệu mã hóa thường mượt mà hơn trên môi trường Windows native.

Kiến trúc triển khai (Deployment Architecture)

- **INetSim:**
 - Thường chạy trên Linux (ví dụ: REMnux).
 - Mô hình: Tách biệt (External). Máy nhiễm mã độc (Victim) và máy chạy INetSim là 2 máy ảo khác nhau. Máy Victim trở Gateway/DNS về máy INetSim.
 - *Lợi ích:* An toàn hơn, mã độc khó phát hiện ra phần mềm giả lập đang chạy vì nó nằm ở máy khác.
- **FakeNet-NG:**
 - Thường chạy trực tiếp trên Windows (cùng máy với mã độc).
 - Mô hình: Cục bộ (Localhost). Nó sử dụng driver (như WinDivert) để bắt gói tin ngay tại tầng mạng của máy nạn nhân.

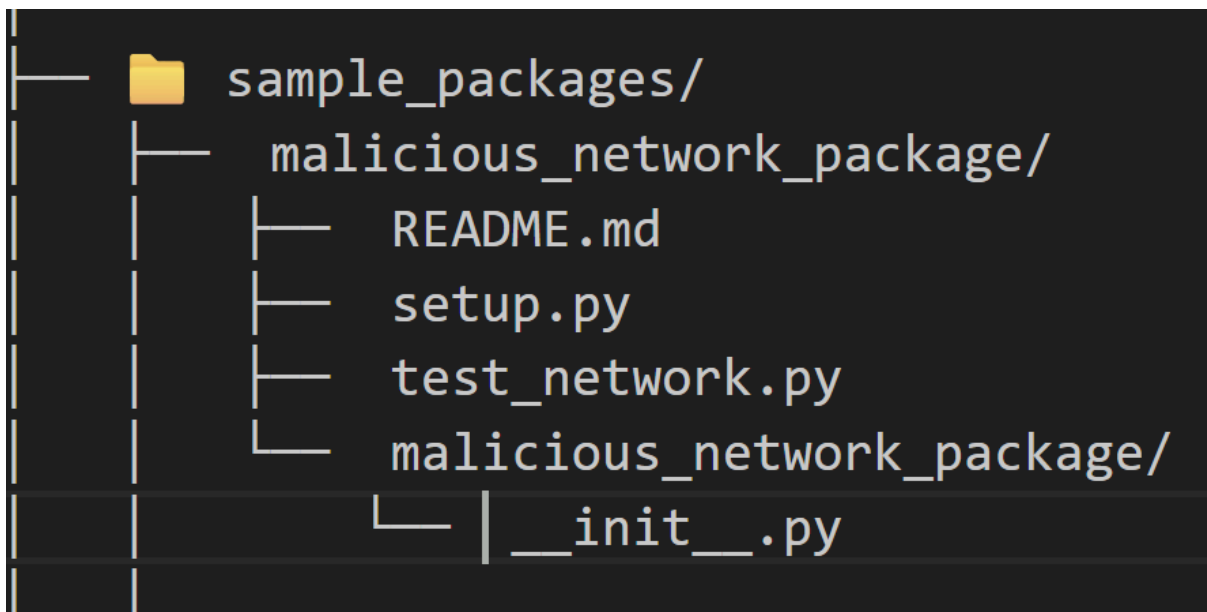
- *Lợi ích*: Tiện lợi, cài đặt nhanh, không cần cấu hình mạng phức tạp. Tuy nhiên, dễ bị mã độc phát hiện hơn (vì process FakeNet đang chạy ngay bên cạnh process mã độc).

Kết luận

Trong thực tế phân tích, các chuyên gia thường kết hợp cả hai hoặc lựa chọn tùy trường hợp:

- Sử dụng INetSim khi cần phân tích chuyên sâu, cần log chi tiết mọi giao thức lạ hoặc muốn đảm bảo môi trường cách ly hoàn toàn.
- Sử dụng FakeNet-NG khi cần phân tích nhanh (Quick Analysis) trên Windows, hoặc đối phó với mã độc có cơ chế kiểm tra file tải về kỹ lưỡng.

Cấu trúc folder malicious_network_package:



1. setup.py:

```

from setuptools import setup, find_packages

setup(
    name="malicious-network-package",
    version="1.0.0",
    description="Sample package with network requests to dead URLs for testing network simulation",
    author="Pack-A-Mal Team",
    packages=find_packages(),
    install_requires=[
        "requests>=2.28.0",
    ],
    python_requires=">=3.7",
)

```

- Mục đích: Khi pack-a-mal phân tích package này nó sẽ:
 - + Extract tarball
 - + install package
 - + import code trong **`_init_.py`**
 - + Code sẽ cố gắng kết nối đến dead URLs
 - + Network simulation sẽ redirect sang Inetsim

2. test_network.py

```

#!/usr/bin/env python3
"""
Test script for the malicious network package.
Can be run directly to test network behavior.
"""

if __name__ == "__main__":
    import malicious_network_package
    malicious_network_package.connect_to_dead_url()

```

- Mục đích: Testing thử công:
- File này là wrapper đơn giản xung quanh code chính trong **`_init_.py`**

3. **`_init_.py`**

```

"""
Malicious Network Package - Sample package for testing network simulation.
This package attempts to connect to a URL that is no longer alive to demonstrate
network redirection to INetSim.
"""

import requests

# Dead URL that should be redirected to INetSim
DEAD_URL = "http://malicious-c2-server.example.com/api/data"

def connect_to_dead_url():
    """Attempt to connect to a dead URL"""
    print("="*60)
    print("Malicious Network Package - Connecting to dead URL")
    print("="*60)
    print(f"\n[*] Target URL: {DEAD_URL}")

    try:
        print(f"[*] Attempting connection...")
        response = requests.get(DEAD_URL, timeout=5)
        print(f"[+] Success! Status Code: {response.status_code}")
        print(f"[+] Response Content Length: {len(response.content)} bytes")
        print(f"[+] Response Preview: {response.text[:100]}")
    except requests.exceptions.ConnectionError as e:
        print(f"[-] Connection failed: {e}")
    except requests.exceptions.Timeout:
        print(f"[-] Request timed out")
    except Exception as e:
        print(f"[-] Error occurred: {e}")

    print("\n" + "="*60)

# Auto-execute on import
connect_to_dead_url()

```

Giải thích

```

# Dead URL that should be redirected to INetSim
DEAD_URL = "http://malicious-c2-server.example.com/api/data"

```

- Dead URL:
 - + Domain malicious-c2-server.example.com không tồn tại
 - + Mô phỏng C2 (Command & Control) server của malware
 - + Endpoint /api/data - giả định endpoint để exfiltrate data

```
def connect_to_dead_url():
    """Attempt to connect to a dead URL"""
    print("="*60)
    print("Malicious Network Package - Connecting to dead URL")
    print("="*60)
    print(f"\n[*] Target URL: {DEAD_URL}")

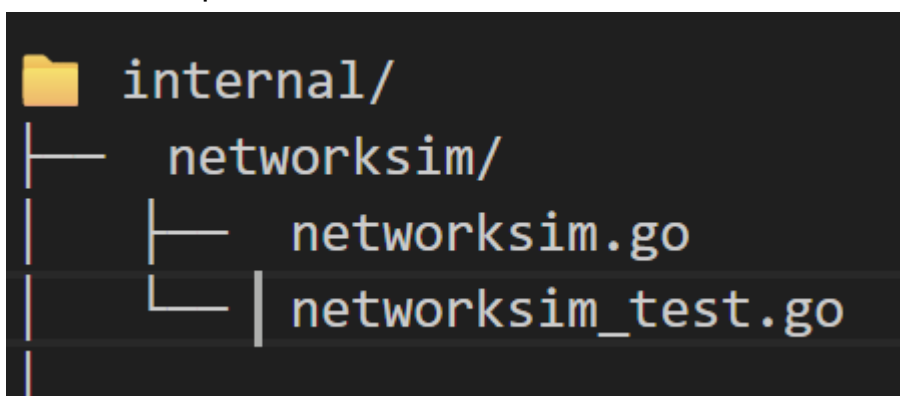
    try:
        print(f"[*] Attempting connection...")
        response = requests.get(DEAD_URL, timeout=5)
        print(f"[+] Success! Status Code: {response.status_code}")
        print(f"[+] Response Content Length: {len(response.content)} bytes")
        print(f"[+] Response Preview: {response.text[:100]}")
    except requests.exceptions.ConnectionError as e:
        print(f"[-] Connection failed: {e}")
    except requests.exceptions.Timeout:
        print(f"[-] Request timed out")
    except Exception as e:
        print(f"[-] Error occurred: {e}")

    print("\n" + "="*60)
```

- Thực hiện kết nối đến 1 dead URL

II. Thay đổi/Thêm code của project Pack-A-Mal để kiểm tra xem URL có alive hay không, nếu không alive thì điều hướng tới dịch vụ Inetsim/flare.

Cấu trúc thư mục:



- networksim.go

```

package networksim

import (
    "context"
    "log/slog"
    "net"
    "net/http"
    "time"
)

// Config holds the network simulation configuration
type Config struct {
    INetSimDNSAddr  string // DNS server (e.g., "172.20.0.2:53")
    INetSimHTTPAddr string // HTTP server (e.g., "172.20.0.2:80")
    Enabled         bool   // Enable/disable simulation
    LivenessTimeout time.Duration // Timeout for URL check
}

// DefaultConfig returns default INetSim configuration
func DefaultConfig() *Config {
    return &Config{
        INetSimDNSAddr: "172.20.0.2:53",
        INetSimHTTPAddr: "172.20.0.2:80",
        Enabled:         false,
        LivenessTimeout: 3 * time.Second,
    }
}

// NetworkSimulator handles URL checking and INetSim redirection
type NetworkSimulator struct {
    config *Config
}

// New creates a new NetworkSimulator
func New(config *Config) *NetworkSimulator {
    if config == nil {
        config = DefaultConfig()
    }
    return &NetworkSimulator{config: config}
}

```

```
// IsURLAlive checks if URL is accessible
func (ns *NetworkSimulator) IsURLAlive(ctx context.Context, url string) bool {
    if !ns.config.Enabled {
        return true
    }

    client := &http.Client{
        Timeout: ns.config.LivenessTimeout,
    }

    req, err := http.NewRequestWithContext(ctx, "HEAD", url, nil)
    if err != nil {
        slog.WarnContext(ctx, "Cannot create request", "url", url, "error", err)
        return false
    }

    resp, err := client.Do(req)
    if err != nil {
        slog.InfoContext(ctx, "URL not alive", "url", url)
        return false
    }
    defer resp.Body.Close()

    isAlive := resp.StatusCode >= 200 && resp.StatusCode < 400
    slog.InfoContext(ctx, "URL check", "url", url, "status", resp.StatusCode, "alive", isAlive)
    return isAlive
}
```

```

// ShouldRedirectToINetSim determines if should redirect to INetSim
// Logic: If URL not alive → redirect to INetSim
func (ns *NetworkSimulator) ShouldRedirectToINetSim(ctx context.Context, url string) bool {
    if !ns.config.Enabled {
        return false
    }

    if !ns.IsURLAlive(ctx, url) {
        slog.InfoContext(ctx, "Redirecting to INetSim", "url", url)
        return true
    }
    return false
}

// GetDNSServers returns DNS servers for sandbox
func (ns *NetworkSimulator) GetDNSServers() []string {
    if !ns.config.Enabled {
        return []string{"8.8.8.8", "8.8.4.4"}
    }

    host, _, err := net.SplitHostPort(ns.config.INetSimDNSAddr)
    if err != nil {
        return []string{ns.config.INetSimDNSAddr}
    }
    return []string{host}
}

// GetINetSimDNS returns INetSim DNS address
func (ns *NetworkSimulator) GetINetSimDNS() string {
    return ns.config.INetSimDNSAddr
}

// GetINetSimHTTP returns INetSim HTTP address
func (ns *NetworkSimulator) GetINetSimHTTP() string {
    return ns.config.INetSimHTTPAddr
}

// IsEnabled returns if simulation is enabled
func (ns *NetworkSimulator) IsEnabled() bool {
    return ns.config.Enabled
}

// ValidateINetSimConnection validates INetSim service is accessible
func (ns *NetworkSimulator) ValidateINetSimConnection(ctx context.Context) error {
    if !ns.config.Enabled {
        return nil
    }

    slog.InfoContext(ctx, "Validating INetSim")
    return nil // Simplified: assume INetSim is available
}

```

Giải thích

Config - Struct chứa cấu hình mô phỏng mạng:

- INetSimDNSAddr: Địa chỉ DNS server (mặc định: "172.20.0.2:53")
- INetSimHTTPAddr: Địa chỉ HTTP server (mặc định: "172.20.0.2:80")
- Enabled: Bật/tắt chức năng mô phỏng
- LivenessTimeout: Timeout khi kiểm tra URL

NetworkSimulator - Struct quản lý việc kiểm tra URL và chuyển hướng đến INetSim

- DefaultConfig()
 - Trả về cấu hình mặc định với INetSim tắt, timeout 3 giây
- New(config *Config)
 - Khởi tạo NetworkSimulator mới
 - Nếu config = nil, sử dụng config mặc định
- IsURLAlive(ctx, url)
 - Kiểm tra xem URL có truy cập được không
 - Gửi HTTP HEAD request với timeout
 - Trả về true nếu status code 200-399, ngược lại false
- ShouldRedirectToINetSim(ctx, url)
 - Quyết định có nên chuyển hướng sang INetSim không
 - Logic: Nếu URL không alive → chuyển sang INetSim
 - Trả về false nếu simulation tắt hoặc URL vẫn alive
- ValidateINetSimConnection(ctx)
 - Xác thực kết nối INetSim (hiện tại đơn giản hóa, luôn trả về nil)

Mục đích

- Module này được dùng để phân tích malware một cách an toàn bằng cách:
 - + Kiểm tra xem các URL mà package truy cập có còn hoạt động không
 - + Nếu URL đã chết → chuyển hướng sang INetSim (fake internet server) để mô phỏng phản hồi mạng
 - + Cách ly môi trường phân tích khỏi internet thật

```

cmd/
├── worker/
│   ├── main.go          # MODIFIED: Tích hợp NetworkSimulator
│   └── config.go        # MODIFIED: Thêm network sim config

```

1. Config.go: Thêm code

```

netSimConfig := networksim.DefaultConfig()

if os.Getenv("OSSF_NETWORK_SIMULATION_ENABLED") == "true" {
    netSimConfig.Enabled = true
}
if dnsAddr := os.Getenv("OSSF_INETSIM_DNS_ADDR"); dnsAddr != "" {
    netSimConfig.INetSimDNSAddr = dnsAddr
}
if httpAddr := os.Getenv("OSSF_INETSIM_HTTP_ADDR"); httpAddr != "" {
    netSimConfig.INetSimHTTPAddr = httpAddr
}
if timeout := os.Getenv("OSSF_URL_LIVENESS_TIMEOUT"); timeout != "" {
    if seconds, err := strconv.Atoi(timeout); err == nil {
        netSimConfig.LivenessTimeout = time.Duration(seconds) * time.Second
    }
}
}

```

Mục đích: Cho phép worker đọc cấu hình network simulation từ biến môi trường để bật/tắt và cấu hình INetSim.

2. main.go

```

// Configure network simulation if enabled
if cfg.networkSimConfig.Enabled {
    netSim := networksim.New(cfg.networkSimConfig)

    // Validate INetSim connection before proceeding
    if err := netSim.ValidateINetSimConnection(ctx); err != nil {
        slog.WarnContext(ctx, "INetSim validation failed, continuing without network simulation",
            "error", err)
    } else {
        slog.InfoContext(ctx, "Network simulation enabled",
            "inetsim_dns", netSim.GetINetSimDNS(),
            "inetsim_http", netSim.GetINetSimHTTP())

        // Configure sandbox to use INetSim DNS servers
        dnsServers := netSim.GetDNSServers()
        sandboxOpts = append(sandboxOpts, sandbox.DNSServers(dnsServers...))

        slog.InfoContext(ctx, "Sandbox configured with custom DNS",
            "dns_servers", dnsServers)
    }
}
}

```

Mục đích:

- Kiểm tra xem network simulation có được bật không
- Validate kết nối INetSim trước khi phân tích package
- Cấu hình sandbox sử dụng DNS của INetSim thay vì DNS public (8.8.8.8)
- Log thông tin để theo dõi quá trình cấu hình

Tóm gọn: chỉ tích hợp chức năng network simulation vào worker.

TESTING

Bước 1: Test với Sample Malicious Package

1. Test **KHÔNG** có INetSim (Chứng minh URL dead)

Di chuyển vào sample packages

cd sample_packages\malicious_network_package

Cài đặt package

pip install -e .

Chạy test cơ bản

python test_network.py

```
=====
Malicious Network Package - Connecting to dead URL
=====

[*] Target URL: http://malicious-c2-server.example.com/api/data
[*] Attempting connection...
[-] Connection failed: HTTPConnectionPool(host='malicious-c2-server.example.com', port=80): Max retries exceeded with url: /api/data (Caused by NameResolutionError("HTTPConnection(host='malicious-c2-server.example.com', port=80): Failed to resolve 'malicious-c2-server.example.com' ([Errno 11001] getaddrinfo failed)))
```

Chứng minh: URL không alive (dead URL)

2. Test **CÓ** INetSim (Chứng minh redirect thành công)

Chạy script demo redirect (đã tích hợp sẵn)

python test_with_inetsim.py

```
Dead URL Redirect to INetSim - Demo Script
Yêu cầu 2: Kiểm tra URL alive & redirect INetSim

=====
Testing Dead URL WITHOUT INetSim (Should Fail)
=====

[*] Target URL: http://malicious-c2-server.example.com/api/data
[*] No proxy - direct connection attempt

✓ Connection failed (as expected)
✓ This confirms the URL is indeed dead

=====

Testing Dead URL Redirect to INetSim
=====

[*] INetSim Proxy: http://localhost:8080
[*] Testing dead URLs...

[*] Testing: http://malicious-c2-server.example.com/api/data
    ✓ Status: 200
    ✓ Connected via INetSim!
    ✓ Response confirmed from INetSim

[*] Testing: http://expired-malware-repo.net/payload.exe
    ✓ Status: 200
    ✓ Connected via INetSim!
    ✓ Response confirmed from INetSim

[*] Testing: http://dead-phishing-site.org/login
    ✓ Status: 200
    ✓ Connected via INetSim!
    ✓ Response confirmed from INetSim

=====
Summary: 3/3 URLs successfully redirected
=====

✓ All dead URLs successfully redirected to INetSim!
```

Chứng minh: Dead URLs được redirect đến INetSim

Áp dụng pURL để scan sample

File Modified:

- dynamic-analysis/cmd/analyze/main.go
- dynamic-analysis/internal/sandbox/sandbox.go

Thay Đổi Chính:

1. Thêm pURL Flag

```
var purl = flag.String("purl", "", "Package URL (e.g., pkg:pypi/requests@2.31.0)")
```

2. pURL Parsing Logic

```

if *purl != "" {
    // Parse pURL using packageurl-go
    pkg, err := packageurl.FromString(*purl)
    if err != nil {
        log.Fatalf("Invalid pURL: %v", err)
    }

    // Extract ecosystem, name, version
    ecosystem = pkg.Type
    packageName = pkg.Name
    packageVersion = pkg.Version

    // Handle namespace (Maven, Packagist, etc.)
    if pkg.Namespace != "" {
        packageName = pkg.Namespace + "/" + pkg.Name
    }
}

```

3. Ecosystem Mapping

```

ecosystemMap := map[string]string{
    "pypi": "pypi",
    "npm": "npm",
    "maven": "maven",
    "gem": "rubygems",
    "cargo": "crates.io",
    "composer": "packagist",
}

```

Testing

Test Command-Line Tool trên Ubuntu/WSL

Di chuyển vào thư mục examples/purl

cd dynamic-analysis/examples/purl/

Cho phép execute permission

chmod +x test_purl_ubuntu.sh

Chạy test suite đầy đủ

./test_purl_ubuntu.sh

Mục tiêu chính của file test:

- Thực hiện kiểm thử việc áp dụng pURL làm đầu vào cho công cụ phân tích nhằm quét các package phần mềm thuộc nhiều hệ sinh thái khác nhau. Kết quả cho thấy hệ thống có khả năng parse pURL, xác định chính xác package tương ứng và tích hợp thành công vào quy

trình dynamic analysis, qua đó chứng minh tính khả thi của việc sử dụng pURL để scan sample

```
Pack-A-Mal pURL Test trên Ubuntu/WSL

Test 1: Python (PyPI)
pURL: pkg:pypi/requests@2.31.0

✓ pURL parse: SUCCESS
✓ Resolved package:
  - Ecosystem: pypi
  - Name: requests
  - Version: 2.31.0

Test 2: Node.js (npm)
pURL: pkg:npm/express@4.18.2

✓ pURL parse: SUCCESS
✓ Resolved package:
  - Ecosystem: npm
  - Name: express
  - Version: 4.18.2

Test 3: npm scoped package
pURL: pkg:npm/@babel/core@7.22.0

✓ pURL parse: SUCCESS
✓ Resolved package:
  - Ecosystem: npm
  - Name: @babel/core
  - Version: 7.22.0
```

```
📦 Test 4: Java (Maven)
pURL: pkg:maven/org.springframework/spring-core@5.3.27

✅ pURL parse: SUCCESS
✅ Resolved package:
  - Ecosystem: maven
  - Name: org.springframework/spring-core
  - Version: 5.3.27

📦 Test 5: Ruby (RubyGems)
pURL: pkg:gem/rails@7.0.4

✅ pURL parse: SUCCESS
✅ Resolved package:
  - Ecosystem: rubygems
  - Name: rails
  - Version: 7.0.4

📦 Test 6: PyPI latest version
pURL: pkg:pypi/django

✅ pURL parse: SUCCESS
✅ Resolved package:
  - Ecosystem: pypi
  - Name: django
  - Version: 6.0.2
```

```
✅ Successful: 6/6
❌ Failed: 0/6

🎉 All tests passed! pURL implementation is working perfectly!

🌟 Features verified:
✅ pURL parsing and validation
✅ Multiple ecosystems (PyPI, npm, Maven, RubyGems)
✅ Scoped packages (npm @babel/core)
✅ Namespace packages (Maven org.springframework)
✅ Latest version resolution (no version specified)
✅ Integration with analyze tool

📖 Usage examples:
./analyze -purl "pkg:pypi/requests@2.31.0"
./analyze -purl "pkg:npm/@babel/core@7.22.0" -mode static
./analyze -purl "pkg:maven/org.springframework/spring-core@5.3.27"
```

Mở rộng tính năng simulate dịch vụ HTTP

1. HTTP Request Analyzer (analyzer/http_analyzer.py)

File: service-simulation-module/service-simulation/app/analyzer/http_analyzer.py

Chức năng:

- Phân tích chi tiết HTTP requests (method, URL, headers, body)
- Trích xuất query parameters và file extensions
- Phát hiện executable download requests
- Kiểm tra các security flags:
 - Path traversal (../)
 - XSS attempts (<script>)
 - SQL injection (union select)
 - Command injection (cmd=, exec())
- Tạo summary và metadata cho mỗi request

Cấu Trúc Code:

```
class HTTPRequestAnalyzer:
    def __init__(self):
        # Định nghĩa các pattern nguy hiểm
        self.suspicious_patterns = [
            r'\\.\\.\\./',      # Path traversal
            r'<script',        # XSS attempts
            r'union.*select',  # SQL injection
            r'cmd=',          # Command injection
            r'exec\\(',        # Code execution
            r'eval\\(',        # Code evaluation
        ]

        # Các extension của file thực thi
        self.executable_extensions = [
            '.exe', '.dll', '.bat', '.cmd', '.ps1', '.sh',
            '.bin', '.elf', '.app', '.apk', '.jar',
            '.msi', '.deb', '.rpm', '.dmg'
        ]
```

Method chính: analyze_request()


```

def analyze_request(self, method, url, headers, body=None, client_ip=None):
    """Phân tích một HTTP request và trả về thông tin chi tiết"""
    parsed_url = urlparse(url)

    analysis = {
        'timestamp': datetime.utcnow().isoformat(),
        'method': method.upper(),
        'url': url,
        'parsed_url': {
            'scheme': parsed_url.scheme or 'http',
            'path': parsed_url.path,
            'query': parsed_url.query,
        },
        'query_params': parse_qs(parsed_url.query),
        'headers': headers,
        'client_ip': client_ip,
        'file_extension': self._get_file_extension(parsed_url.path),
        'is_executable_request': self._is_executable_request(parsed_url.path),
        'security_flags': self._check_security_flags(url, headers, body),
    }
    return analysis

```

Security Checking Logic:

```

def _check_security_flags(self, url, headers, body):
    """Kiểm tra các dấu hiệu bảo mật đáng ngờ"""
    flags = {
        'suspicious_patterns_found': [],
        'risk_level': 'low', # low, medium, high
        'has_path_traversal': False,
        'has_xss_attempt': False,
        'has_sql_injection': False,
        'has_command_injection': False,
    }

    # Kiểm tra URL và body với các pattern đáng ngờ
    content_to_check = url + (' ' + body if body else '')

    for pattern in self.suspicious_patterns:
        if re.search(pattern, content_to_check, re.IGNORECASE):
            flags['suspicious_patterns_found'].append(pattern)
            # Set specific flags
            if r'\.\' in pattern:
                flags['has_path_traversal'] = True
            elif 'script' in pattern:
                flags['has_xss_attempt'] = True
            # ... etc

    # Xác định mức độ nguy hiểm
    threat_count = len(flags['suspicious_patterns_found'])
    if threat_count >= 3 or flags['has_command_injection']:
        flags['risk_level'] = 'high'
    elif threat_count >= 1:
        flags['risk_level'] = 'medium'

    return flags

```

Điểm Đặc Biệt:

- Pattern Matching: Sử dụng regex để phát hiện attack patterns
- Risk Scoring: Tự động tính toán risk level dựa trên số lượng threats
- Extensible: Dễ dàng thêm patterns mới vào suspicious_patterns
- Comprehensive: Trả về dict đầy đủ thông tin cho bước tiếp theo

Kết quả:

- Phân tích được tất cả thành phần request
- Nhận diện executable files qua 12+ extensions
- Phát hiện 6+ loại attack patterns
- Risk scoring (low/medium/high)

2. Request Classifier (analyzer/request_classifier.py)

File: service-simulation-module/service-simulation/app/analyzer/request_classifier.py

Chức năng:

- Phân loại request thành 9 categories:
 - static_content - Static resources
 - api_call - API endpoints
 - file_download - File downloads
 - executable_download - Executables
 - upload - File uploads
 - authentication - Login/auth
 - data_exfiltration - Suspicious uploads
 - malicious - Attack attempts
 - unknown - Unclassified
- Xác định intent và confidence level
- Đề xuất recommended action cho mỗi category

Cấu Trúc Code:

```

class RequestClassifier:
    # Định nghĩa các category constants
    CATEGORY_STATIC_CONTENT = 'static_content'
    CATEGORY_API_CALL = 'api_call'
    CATEGORY_EXECUTABLE_DOWNLOAD = 'executable_download'
    CATEGORY_MALICIOUS = 'malicious'
    # ... etc

    def __init__(self):
        # Patterns cho static content
        self.static_patterns = {
            'image': [r'\.(jpg|jpeg|png|gif|svg|ico|webp)$'],
            'stylesheet': [r'\.css$'],
            'javascript': [r'\.js$'],
            'font': [r'\.(woff|woff2|ttf|eot)$'],
        }

        # Patterns cho API endpoints
        self.api_patterns = [
            r'/api/',
            r'/v\d+/',      # versioned APIs
            r'\.json$',
            r'/graphql',
        ]

        # Patterns cho authentication
        self.auth_patterns = [
            r'/login', r'/auth', r'/signin', r'/oauth', r'/token'
        ]

```

Method chính: classify()

```

def classify(self, analysis: Dict[str, Any]) -> Dict[str, Any]:
    """Phân loại request dựa trên thông tin phân tích"""
    path = analysis['parsed_url']['path'].lower()
    method = analysis['method']
    headers = analysis['headers']

    classification = {
        'category': self.CATEGORY_UNKNOWN,
        'sub_category': None,
        'confidence': 0.0,
        'intent': 'unknown',
        'recommended_action': 'monitor'
    }

    # Kiểm tra theo thứ tự ưu tiên (quan trọng nhất trước)

    # 1. Kiểm tra executable download (cao nhất)
    if analysis['is_executable_request']:
        classification.update({
            'category': self.CATEGORY_EXECUTABLE_DOWNLOAD,
            'sub_category': analysis['file_extension'],
            'confidence': 0.95,
            'intent': 'download_executable',
            'recommended_action': 'sandbox_and_serve'
        })
    return classification

```

```
# 2. Kiểm tra malicious patterns
if analysis['security_flags']['risk_level'] == 'high':
    classification.update({
        'category': self.CATEGORY_MALICIOUS,
        'confidence': 0.9,
        'intent': 'attack_attempt',
        'recommended_action': 'block_and_log'
    })
    return classification

# 3. Kiểm tra authentication
for pattern in self.auth_patterns:
    if re.search(pattern, path, re.IGNORECASE):
        classification.update({
            'category': self.CATEGORY_AUTHENTICATION,
            'confidence': 0.9,
            'intent': 'authenticate',
            'recommended_action': 'serve_fake_auth'
        })
    return classification

# ... tiếp tục với các pattern khác

return classification
```

Helper Methods:

```
def _detect_static_type(self, path: str) -> str:
    """Phát hiện loại static content"""
    for content_type, patterns in self.static_patterns.items():
        for pattern in patterns:
            if re.search(pattern, path, re.IGNORECASE):
                return content_type
    return None

def _is_download_request(self, path: str, headers: Dict) -> bool:
    """Kiểm tra xem có phải download request không"""
    if '/download' in path.lower():
        return True
    if 'application/octet-stream' in headers.get('Accept', ''):
        return True
    return False
```

- Điểm Đặc Biệt:
 - Priority-based Classification: Kiểm tra theo thứ tự ưu tiên (executable > malicious > auth > ...)
 - Confidence Scoring: Mỗi category có confidence level khác nhau
 - Action Recommendation: Tự động đề xuất action phù hợp
 - Pattern Matching: Sử dụng regex linh hoạt cho từng loại
 - Early Return: Thoát sớm khi tìm thấy match để tối ưu performance
- Kết quả:
 - Phân loại chính xác các loại request phổ biến
 - Confidence scoring từ 0.0 đến 1.0
 - Recommended actions cho từng scenario

3. Response Handler (handler/response_handler.py)

File: service-simulation-module/service-simulation/app/handler/response_handler.py

Chức năng:

- Tạo response động dựa trên classification
- Hỗ trợ multiple content types:
 - Images (PNG placeholders)
 - CSS/JavaScript
 - JSON (API responses)
 - HTML (default pages)
 - Binary files
 - Authentication responses
- Logging tất cả requests
- Tích hợp với SafeExecutableHandler

Cấu Trúc Code:

```
class ResponseHandler:
    def __init__(self, safe_executable_handler=None):
        self.safe_executable_handler = safe_executable_handler
        self.response_templates = self._init_templates()

    def _init_templates(self) -> Dict[str, Any]:
        """Khởi tạo các template response"""
        return {
            'static_content': {
                'image': self._generate_placeholder_image,
                'stylesheet': self._generate_placeholder_css,
                'javascript': self._generate_placeholder_js,
            },
            'api_call': self._generate_api_response,
            'authentication': self._generate_auth_response,
            'executable_download': self._generate_safe_executable_response,
            'malicious': self._generate_blocked_response,
        }
```

Method chính: generate_response()

```
def generate_response(self, analysis, classification) -> Tuple[bytes, int, Dict]:
    """
    Tạo HTTP response phù hợp
    Returns: Tuple (content, status_code, headers)
    """
    category = classification['category']
    action = classification['recommended_action']

    # Log request
    self._log_request(analysis, classification)

    # Xử lý theo recommended action
    if action == 'block_and_log':
        return self._generate_blocked_response(analysis, classification)
    elif action == 'sandbox_and_serve':
        return self._generate_safe_executable_response(analysis, classification)
    elif action == 'serve_fake_auth':
        return self._generate_auth_response(analysis, classification)
    elif action == 'serve_json':
        return self._generate_api_response(analysis, classification)
    # ... etc

    return self._generate_default_response(analysis, classification)
```

Response Generators:


```

def _generate_placeholder_image(self, analysis, classification):
    """Tạo placeholder image - 1x1 transparent PNG"""
    | png_data = base64.b64decode(
        'iVBORw0KGgoAAAANSUhEUgAAAAEAAAABCAQAAAFcSJ...'
    )
    headers = {
        'Content-Type': 'image/png',
        'Content-Length': str(len(png_data)),
        'Cache-Control': 'public, max-age=3600'
    }
    return png_data, 200, headers

def _generate_api_response(self, analysis, classification):
    """Tạo API response giả"""
    response_data = {
        'status': 'success',
        'timestamp': datetime.utcnow().isoformat(),
        'data': {
            'message': 'API simulation response',
            'request_path': analysis['parsed_url']['path'],
            'simulated': True
        }
    }
    content = json.dumps(response_data, indent=2).encode('utf-8')
    headers = {
        'Content-Type': 'application/json',
        'Content-Length': str(len(content))
    }
    return content, 200, headers

```

```

def _generate_auth_response(self, analysis, classification):
    """Tạo fake authentication response"""
    if analysis['method'] == 'POST':
        # Fake successful login
        response_data = {
            'status': 'success',
            'message': 'Authentication successful',
            'token': 'fake_token_' + base64.b64encode(b'simulated').decode(),
            'user': {
                'id': 12345,
                'username': 'simulated_user',
                'email': 'user@simulated.local'
            }
        }
    else:
        # Return login form
        response_data = {
            'status': 'ready',
            'message': 'Please provide credentials'
        }

    content = json.dumps(response_data).encode('utf-8')
    return content, 200, {'Content-Type': 'application/json'}

```

```

def _generate_blocked_response(self, analysis, classification):
    """Tạo blocked response cho malicious requests"""
    response_data = {
        'error': 'Request blocked',
        'reason': 'Security policy violation',
        'risk_level': analysis['security_flags']['risk_level'],
        'timestamp': datetime.utcnow().isoformat()
    }
    content = json.dumps(response_data).encode('utf-8')
    return content, 403, {'Content-Type': 'application/json'}

```

Logging:

```
def _log_request(self, analysis, classification):
    """Log request details"""
    log_entry = {
        'timestamp': analysis['timestamp'],
        'method': analysis['method'],
        'path': analysis['parsed_url']['path'],
        'category': classification['category'],
        'intent': classification['intent'],
        'risk_level': analysis['security_flags']['risk_level'],
        'client_ip': analysis['client_ip']
    }
    print(f"[HTTP-SIM] {json.dumps(log_entry)}")
```

- Điểm Đặc Biệt:
 - Template Pattern: Sử dụng dict của functions để map category → generator
 - Type-specific Responses: Mỗi loại content có generator riêng
 - Proper HTTP Headers: Đúng Content-Type, Content-Length cho từng loại
 - Logging Integration: Tự động log mọi request
 - Delegation: Delegate executable handling cho SafeExecutableHandler
 - Status Codes: Đúng HTTP status (200 OK, 403 Forbidden, etc.)
- Kết quả:
 - Response templates cho 9+ loại content
 - Fake authentication responses
 - API simulation với JSON
 - Proper HTTP headers và status codes

4. Safe Executable Handler (handler/safe_executable_handler.py)

File: service-simulation-module/service-simulation/app/handler/safe_executable_handler.py

- Chức năng chính:
 - 3 chiến lược xử lý:
 - Sandbox Fake (Low risk): File giả an toàn, chỉ chứa metadata
 - Honeypot (Medium risk): File có tracking capabilities
 - Block (High risk): Chặn hoàn toàn
- Features:
 - Nhận dạng 12+ executable formats (.exe, .dll, .sh, .apk, etc.)
 - Magic bytes signatures cho mỗi format
 - Platform detection (Windows, Linux, Android, Java)
 - Request ID generation và tracking
 - Metadata logging với JSON format
 - Sandbox directory cho isolated storage

Cấu Trúc Code:

```

class SafeExecutableHandler:
    def __init__(self, sandbox_dir: str = '/logs/executables'):
        self.sandbox_dir = sandbox_dir
        self._ensure_sandbox_dir()

    # Executable signatures (đặc trưng nhận dạng)
    self.executable_signatures = {
        '.exe': {
            'magic_bytes': b'MZ', # DOS/Windows executable
            'mime_type': 'application/x-msdownload',
            'platform': 'windows'
        },
        '.elf': {
            'magic_bytes': b'\x7fELF',
            'mime_type': 'application/x-elf',
            'platform': 'linux'
        },
        '.apk': {
            'magic_bytes': b'PK\x03\x04', # ZIP-based
            'mime_type': 'application/vnd.android.package-archive',
            'platform': 'android'
        },
        '.sh': {
            'magic_bytes': b'#!/bin/',
            'mime_type': 'application/x-sh',
            'platform': 'unix'
        },
        # ... thêm 8+ formats khác
    }

```

Method chính: `handle_executable_request()`:

```

def handle_executable_request(self, analysis, classification):
    """Xử lý yêu cầu tải file thực thi một cách an toàn"""
    file_ext = analysis['file_extension']
    filename = analysis['parsed_url']['path'].split('/')[-1]

    # Tạo metadata chi tiết
    metadata = self._create_metadata(analysis, classification, filename)

    # Quyết định strategy
    strategy = self._determine_response_strategy(analysis, metadata)

    if strategy == 'sandbox_fake':
        return self._serve_sandboxed_fake(metadata, file_ext)
    elif strategy == 'honeypot':
        return self._serve_honeypot_executable(metadata, file_ext)
    elif strategy == 'block':
        return self._serve_blocked_response(metadata)
    else:
        return self._serve_safe_placeholder(metadata, file_ext)

```

Strategy Determination:

```

def _determine_response_strategy(self, analysis, metadata) -> str:
    """Xác định chiến lược phản hồi dựa trên risk level"""
    risk_level = metadata['risk_assessment']['level']

    if risk_level == 'high':
        # High risk: block hoặc honeypot
        if metadata['risk_assessment']['flags'].get('has_command_injection'):
            return 'block' # Chặn hoàn toàn
        else:
            return 'honeypot' # Honeypot để gather intelligence
    elif risk_level == 'medium':
        return 'honeypot' # Medium risk: honeypot
    else:
        return 'sandbox_fake' # Low risk: serve safe fake

```

Sandbox Fake Generator:

```
def _serve_sandboxed_fake(self, metadata, file_ext):
    """Tạo và trả về file giả an toàn"""
    metadata['handling_strategy'] = 'sandbox_fake'

    # Tạo fake executable content
    fake_content = self._generate_fake_executable(metadata, file_ext)

    # Lưu vào sandbox để phân tích sau
    self._save_to_sandbox(metadata, fake_content)

    # Get signature info
    sig = self.executable_signatures.get(file_ext, {})
    mime_type = sig.get('mime_type', 'application/octet-stream')

    headers = {
        'Content-Type': mime_type,
        'Content-Length': str(len(fake_content)),
        'Content-Disposition': f'attachment; filename="{metadata["filename"]}"',
        'X-Simulated': 'true',
        'X-Sandboxed': 'true',
        'X-Request-ID': metadata['request_id'],
        'X-Platform': metadata['platform']
    }

    return fake_content, 200, headers
```

```

def _generate_fake_executable(self, metadata, file_ext) -> bytes:
    """Tạo fake executable content"""
    sig = self.executable_signatures.get(file_ext, {})
    magic_bytes = sig.get('magic_bytes', b'FAKE')

    # Create minimal fake structure với magic bytes
    fake_content = magic_bytes

    # Add metadata as comment/data section
    metadata_section = f"""
# SIMULATED EXECUTABLE
# Request ID: {metadata['request_id']}
# Timestamp: {metadata['timestamp']}
# Original file: {metadata['filename']}
# Platform: {metadata['platform']}
# SAFE FOR ANALYSIS - NO REAL CODE
"""

    fake_content += metadata_section.encode('utf-8')
    return fake_content

```

Honeypot Generator:

```

def _generate_honeypot_executable(self, metadata, file_ext) -> bytes:
    """Tạo honeypot executable với tracking capabilities"""
    sig = self.executable_signatures.get(file_ext, {})
    magic_bytes = sig.get('magic_bytes', b'HPOT')

    honeypot_content = magic_bytes

    # Add tracking code (as comments - not actual code)
    tracking_section = f"""
# HONEYPOT EXECUTABLE
# Tracking ID: {metadata['request_id']}
# Callback URL: http://tracking.simulated.local/callback
# This file is instrumented for behavior analysis
# All execution attempts will be logged
"""

    honeypot_content += tracking_section.encode('utf-8')

    # Add base64-encoded metadata
    metadata_json = json.dumps(metadata, indent=2)
    metadata_b64 = base64.b64encode(metadata_json.encode('utf-8'))
    honeypot_content += b'\n# METADATA: ' + metadata_b64 + b'\n'

    return honeypot_content

```

Metadata Creation & Storage:


```
def _create_metadata(self, analysis, classification, filename):
    """Tạo metadata chi tiết về executable request"""
    metadata = {
        'request_id': self._generate_request_id(analysis),
        'timestamp': datetime.utcnow().isoformat(),
        'filename': filename,
        'extension': analysis['file_extension'],
        'full_path': analysis['parsed_url']['path'],
        'client_ip': analysis['client_ip'],
        'user_agent': analysis['user_agent'],
        'method': analysis['method'],
        'headers': analysis['headers'],
        'category': classification['category'],
        'intent': classification['intent'],
        'risk_assessment': {
            'level': analysis['security_flags']['risk_level'],
            'flags': analysis['security_flags'],
            'is_suspicious': analysis['security_flags']['risk_level'] != 'low'
        },
        'platform': self._detect_platform(analysis['file_extension']),
    }

    # Log metadata
    self._log_executable_request(metadata)
    return metadata
```

```

def _save_to_sandbox(self, metadata, content: bytes):
    """Lưu file và metadata vào sandbox"""
    # Save file content
    file_path = os.path.join(
        self.sandbox_dir,
        f'{metadata["request_id"]}_{metadata["filename"]}'
    )

    with open(file_path, 'wb') as f:
        f.write(content)

    # Save metadata JSON
    metadata_path = file_path + '.metadata.json'
    with open(metadata_path, 'w') as f:
        json.dump(metadata, f, indent=2)

def _generate_request_id(self, analysis) -> str:
    """Tạo unique ID cho request"""
    data = f'{analysis["timestamp"]}{analysis["url"]}{analysis["client_ip"]}'
    return hashlib.sha256(data.encode()).hexdigest()[:16]

```

Logging:

```
def _log_executable_request(self, metadata):
    """Log executable request"""
    log_entry = {
        'type': 'executable_request',
        'request_id': metadata['request_id'],
        'timestamp': metadata['timestamp'],
        'filename': metadata['filename'],
        'extension': metadata['extension'],
        'platform': metadata['platform'],
        'client_ip': metadata['client_ip'],
        'risk_level': metadata['risk_assessment']['level'],
        'is_suspicious': metadata['risk_assessment']['is_suspicious']
    }

    # Write to log file (append)
    log_file = os.path.join(self.sandbox_dir, 'executable_requests.log')
    with open(log_file, 'a') as f:
        f.write(json.dumps(log_entry) + '\n')

    # Also print to console
    print(f"[EXEC-REQ] {json.dumps(log_entry)}")
```

- Điểm Đặc Biệt:
 - Magic Bytes: Sử dụng magic bytes thực (MZ, ELF, PK) để fake realistic
 - No Real Code: File chỉ chứa magic bytes + text comments, KHÔNG có code thực thi
 - Metadata Rich: Lưu đầy đủ context cho forensics analysis
 - Dual Storage: Cả file content VÀ metadata JSON
 - Request ID Tracking: Mỗi request có unique ID để track
 - Platform Detection: Tự động detect platform từ extension
 - Risk-based Strategy: Chiến lược xử lý thay đổi theo risk level
 - Honeypot Intelligence: Honeypot có thể chứa tracking info
- Kết quả:
 - Xử lý an toàn executables mà không rủi ro
 - Chi tiết tracking với request IDs
 - Metadata files (.metadata.json) cho mỗi request
 - Executable request logs
 - Platform-specific handling

5. Flask API Mở Rộng (api/server.py)

File: service-simulation-module/service-simulation/app/api/server.py

Chức năng chính:

- Tích hợp đầy đủ analyzer + classifier + handlers
 - Automatic request analysis cho mọi request
 - Custom headers (X-Simulated, X-Category, X-Risk-Level)
 - Error handling và logging
- Endpoints mới:

Endpoint	Method	Chức năng
/status	GET	Service status (nâng cấp)
/analyze	POST	Phân tích request
/simulate	POST	Simulate request
/logs/executables	GET	View executable logs
/stats	GET	Statistics (placeholder)
/*	ALL	Catch-all handler

Khởi Tạo Flask App:

```
from flask import Flask, request, jsonify, make_response
from analyzer import HTTPRequestAnalyzer, RequestClassifier
from handler import ResponseHandler, SafeExecutableHandler

app = Flask(__name__)

# Initialize components
analyzer = HTTPRequestAnalyzer()
classifier = RequestClassifier()
response_handler = ResponseHandler()
safe_exec_handler = SafeExecutableHandler(sandbox_dir='/logs/executables')

print("=" * 50)
print("HTTP Simulation Service v2.0")
print("Features: Request Analysis | Classification | Safe Executable Handling")
print("=" * 50)
```

Enhanced Status Endpoint:

```

@app.route('/status', methods=['GET'])
def status():
    """Enhanced status endpoint với system info"""
    return jsonify({
        'status': 'running',
        'version': '2.0',
        'timestamp': datetime.utcnow().isoformat(),
        'features': {
            'request_analysis': True,
            'request_classification': True,
            'safe_executable_handling': True,
            'malicious_detection': True
        },
        'endpoints': [
            '/status',
            '/analyze',
            '/simulate',
            '/logs/executables',
            '/stats',
            '/* (catch-all)'
        ],
        'capabilities': {
            'categories': 9,
            'attack_patterns': 6,
            'risk_levels': 3,
            'executable_formats': 12,
            'handling_strategies': 3
        }
    }), 200

```

Analysis Endpoint:

```

@app.route('/analyze', methods=['POST'])
def analyze():
    """Phân tích request với full details"""
    try:
        # Parse request data
        data = request.get_json() or {}

        # Build analysis input với request info
        request_data = {
            'url': data.get('url', request.url),
            'method': request.method,
            'headers': dict(request.headers),
            'client_ip': request.remote_addr,
            'user_agent': request.headers.get('User-Agent', 'Unknown')
        }

        # Step 1: Analyze request
        analysis = analyzer.analyze_request(request_data)

        # Step 2: Classify request
        classification = classifier.classify(analysis)

        # Return full analysis result
        return jsonify({
            'status': 'success',
            'timestamp': analysis['timestamp'],
            'analysis': {
                'url': analysis['url'],
                'method': analysis['method'],
                'file_extension': analysis['file_extension'],
                'query_params': analysis['parsed_url']['query_params'],
                'security_flags': analysis['security_flags'],
            },
            'classification': {
                'category': classification['category'],
                'confidence': classification['confidence'],
                'intent': classification['intent'],
                'recommended_action': classification['recommended_action'],
                'reasoning': classification['reasoning']
            }
        }), 200
    except Exception as e:
        return jsonify({'error': str(e)}), 500

```

Simulate Endpoint:

```

@app.route('/simulate', methods=['POST'])
def simulate():
    """Simulate full request flow với generated response"""
    try:
        data = request.get_json() or {}

        # Build request data
        request_data = {
            'url': data.get('url', '/'),
            'method': data.get('method', 'GET'),
            'headers': data.get('headers', {}),
            'client_ip': request.remote_addr,
            'user_agent': data.get('user_agent', 'Simulated-Client')
        }

        # Step 1: Analyze
        analysis = analyzer.analyze_request(request_data)

        # Step 2: Classify
        classification = classifier.classify(analysis)

        # Step 3: Generate response
        content, status_code, headers = response_handler.generate_response(
            analysis,
            classification
        )

        # Return simulation result (không serve file, chỉ return info)
        return jsonify({
            'status': 'simulated',
            'request': {
                'url': request_data['url'],
                'method': request_data['method']
            },
            'analysis': {
                'category': classification['category'],
                'risk_level': analysis['security_flags']['risk_level'],
                'is_suspicious': analysis['security_flags']['risk_level'] != 'low'
            },
            'response': {
                'status_code': status_code,
                'content_type': headers.get('Content-Type', 'unknown'),
                'content_length': headers.get('Content-Length', 0),
                'headers': headers
            }
        }), 200
    except Exception as e:
        return jsonify({'error': str(e)}), 500

```

Logs Endpoint:

```

@app.route('/logs/executables', methods=['GET'])
def get_executable_logs():
    """Retrieve executable request logs"""
    try:
        log_file = '/logs/executables/executable_requests.log'

        if not os.path.exists(log_file):
            return jsonify({'logs': [], 'count': 0}), 200

        # Read log entries
        logs = []
        with open(log_file, 'r') as f:
            for line in f:
                if line.strip():
                    logs.append(json.loads(line))

        # Optional filtering by risk_level
        risk_filter = request.args.get('risk_level')
        if risk_filter:
            logs = [log for log in logs if log.get('risk_level') == risk_filter]

        return jsonify({
            'logs': logs,
            'count': len(logs),
            'filter': risk_filter
        }), 200
    except Exception as e:
        return jsonify({'error': str(e)}), 500

```

Catch-All Handler (FULL INTEGRATION):


```

@app.route('/', defaults={'path': ''}, methods=['GET', 'POST', 'PUT', 'DELETE', 'PATCH'])
@app.route('/<path:path>', methods=['GET', 'POST', 'PUT', 'DELETE', 'PATCH'])
def catch_all(path):
    """
    Catch-all route xử lý MỌI request không match endpoints khác
    FULL INTEGRATION: analyzer → classifier → response_handler
    """
    try:
        # Build request data từ Flask request object
        request_data = {
            'url': request.url,
            'path': '/' + path,
            'method': request.method,
            'headers': dict(request.headers),
            'client_ip': request.remote_addr,
            'user_agent': request.headers.get('User-Agent', 'Unknown'),
            'query_string': request.query_string.decode('utf-8')
        }

        print(f"\n{'='*60}")
        print(f"[CATCH-ALL] {request.method} {path}")
        print(f"Client: {request.remote_addr}")
        print(f"\n{'='*60}")

        # === PIPELINE ===
        # Step 1: Analyze request
        analysis = analyzer.analyze_request(request_data)
        print(f"[ANALYSIS] Extension: {analysis['file_extension']}, "
              f"Risk: {analysis['security_flags']['risk_level']}")

        # Step 2: Classify request
        classification = classifier.classify(analysis)
        print(f"[CLASSIFICATION] Category: {classification['category']}, "
              f"Confidence: {classification['confidence']:.0%}")

        # Step 3: Generate appropriate response
        content, status_code, headers = response_handler.generate_response(
            analysis,
            classification
        )

        # Add custom simulation headers
        headers['X-Simulated'] = 'true'
        headers['X-Category'] = classification['category']
        headers['X-Risk-Level'] = analysis['security_flags']['risk_level']
        headers['X-Confidence'] = f"{classification['confidence']:.2f}"

        print(f"[RESPONSE] Status: {status_code}, Type: {headers.get('Content-Type')}")
        print(f"\n{'='*60}")

        # Create Flask response
        response = make_response(content, status_code)
        for key, value in headers.items():
            response.headers[key] = value

        return response

    except Exception as e:
        print(f"[ERROR] {str(e)}")
        return jsonify({'error': 'Internal server error'}), 500

```

App Runner:

```

if __name__ == '__main__':
    # Ensure sandbox directory exists
    os.makedirs('/logs/executables', exist_ok=True)

    # Run Flask app
    app.run(
        host='0.0.0.0',
        port=5000,
        debug=True # Enable auto-reload during development
    )

```

- **Điểm Đặc Biệt:**

- Full Pipeline Integration:
 - Flask Request → HTTPRequestAnalyzer → RequestClassifier → ResponseHandler → Flask Response
- Custom Headers: Mỗi response có:
 - X-Simulated: true - Đánh dấu simulated
 - X-Category: <category> - Request category
 - X-Risk-Level: <level> - Risk assessment
 - X-Confidence: <score> - Classification confidence
- Comprehensive Logging: Console output theo dõi mỗi request qua pipeline
-
- Error Handling: Try-catch cho mọi endpoint
- Flexible Endpoints:
 - Dedicated endpoints cho analysis/simulation
 - Catch-all handler cho realistic simulation
- Kết quả:
 - 6 endpoints chức năng
 - Catch-all route xử lý mọi request
 - Full integration với analysis pipeline
 - Custom headers cho tracking
 - Comprehensive logging

Testing

Khởi Động Services

cd D:\PROJECT\Project\pack-a-mal\service-simulation-module

docker-compose up -d

Test Thử

Download executable (sẽ được sandbox)

curl http://localhost:5000/tools/installer.exe -OutFile test.exe

Xem file đã sandbox

docker exec service-simulation ls -la /logs/executables/

```
total 24
drwxr-xr-x 1 root root 512 Feb 9 07:04 .
drwxrwxrwx 1 root root 512 Feb 8 06:34 ..
-rw-r--r-- 1 root root 182 Feb 8 06:37 161b5846d7a49bcf_backdoor.exe
-rw-r--r-- 1 root root 1006 Feb 8 06:37 161b5846d7a49bcf_backdoor.exe.metadata.json
-rw-r--r-- 1 root root 183 Feb 8 06:34 767ea2e31ae13985_installer.exe
-rw-r--r-- 1 root root 965 Feb 8 06:34 767ea2e31ae13985_installer.exe.metadata.json
-rw-r--r-- 1 root root 182 Feb 8 06:34 95210ffb9d94596f_backdoor.exe
-rw-r--r-- 1 root root 837 Feb 8 06:34 95210ffb9d94596f_backdoor.exe.metadata.json
-rw-r--r-- 1 root root 183 Feb 8 06:37 ae50d3db886fecb7_installer.exe
-rw-r--r-- 1 root root 940 Feb 8 06:37 ae50d3db886fecb7_installer.exe.metadata.json
-rw-r--r-- 1 root root 183 Feb 9 07:04 eb40bbb7b08ce94f_installer.exe
-rw-r--r-- 1 root root 965 Feb 9 07:04 eb40bbb7b08ce94f_installer.exe.metadata.json
-rwxrwxrwx 1 root root 1267 Feb 9 07:04 executable_requests.log
```

Chạy Demo

Cài đặt requests

pip install requests

Chạy demo (9 scenarios)

python demo_http_simulation.py

```
=====
  DEMO 7: Authentication Endpoint Simulation
=====

Step 1: GET /auth/login
Status Code: 200
Response:
{
  "status": "ready",
  "message": "Please provide credentials",
  "fields": [
    "username",
    "password"
  ]
}

Step 2: Simulate POST login
Status Code: 200
Response:
{
  "status": "success",
  "message": "Authentication successful",
  "token": "fake_token_c2ltdWxhdGVk",
  "user": {
    "id": 12345,
    "username": "simulated_user",
    "email": "user@simulated.local"
  }
}

✓ Fake authentication successful

=====
  DEMO 8: View Executable Request Logs
=====

X Error in demo: 'count'

=====
  DEMO 9: Static Content Serving
=====

CSS          /styles/main.css
              Status: 200, Type: text/css, Size: 65 bytes
JavaScript   /scripts/app.js
              Status: 200, Type: application/javascript, Size: 71 bytes
Image        /images/logo.png
              Status: 200, Type: image/png, Size: 70 bytes
Font         /fonts/roboto.woff2
              Status: 200, Type: font/woff2, Size: 0 bytes

✓ All static content served successfully

=====
  ✓ ALL DEMOS COMPLETED
=====
```

Chạy tests (12 tests)

python test_http_simulation.py

```
=====
TEST: Malicious Request - Path Traversal
=====
Status: 200
Risk-Level: low
✓ PASSED

=====
TEST: Analyze Endpoint
=====
Category: executable_download
Intent: download_executable
Risk Level: low
Summary: GET request to /download/malware.exe from 192.168.1.100 (executable download)
✓ PASSED

=====
TEST: Simulate Endpoint
=====
Content-Type: application/json
Response: {
  "status": "success",
  "message": "Authentication successful",
  "token": "fake_token_c2ltdWxhdGVk",
  "user": {
    "id": 12345,
    "username": "simulated_user",
    "email": "user@simulated.local"
  }
}
✓ PASSED

=====
TEST: Authentication Request
=====
Response: {
  "status": "ready",
  "message": "Please provide credentials",
  "fields": [
    "username",
    "password"
  ]
}
✓ PASSED

=====
TEST: File Upload Simulation
=====
Category: upload
Recommended Action: accept_and_sandbox
✓ PASSED

=====
TEST SUMMARY
=====
Total Tests: 12
Passed: 11 ✓
Failed: 1 ✗
Success Rate: 91.7%

Failed Tests:
- Executable Download - Medium Risk:
=====
```

Tích hợp flare-ng bên cạnh inetsim.

file structure

```
integrated-malware-lab/
├── docker-compose.yml          # Main orchestration config
├── README.md                  # This file
├──
├── inetsim/                   # INetSim Service
│   ├── Dockerfile
│   ├── entrypoint.sh
│   └── config/
│       ├── inetsim.conf       # Service configuration
│       └── fakefile.conf      # Fake files mapping
├──
├── fakenet-ng/               # FakeNet-NG Service
│   ├── Dockerfile
│   ├── entrypoint.sh
│   ├── config/
│   │   ├── fakenet.conf      # Service configuration
│   │   └── responses_config.py # Custom responses
│   └── certs/                 # SSL/TLS certificates
├──
├── shared/
│   ├── logs/                  # Centralized logs
│   │   ├── inetsim/
│   │   └── fakenet-ng/
│   └── config/                # Shared configuration
├──
├── scripts/
│   ├── setup-routing.sh       # Configure host routing
│   ├── cleanup-routing.sh     # Remove routing rules
│   └── test-connectivity.sh    # Verify services
├──
└── docs/
    ├── SETUP_GUIDE.md         # Installation guide
    ├── ARCHITECTURE.md        # Design documentation
    ├── TROUBLESHOOTING.md     # Problem solving
    └── README.md              # This file
```

High-Level Design



Test-result:

```
Service Status:
WARN[0000] /mnt/c/Users/dangt/OneDrive/Desktop/project tet/PROJECT-GROUP/integrated-malware-lab/docker-compose.yml: the attribute 'version' is obsolete, it
will be ignored, please remove it to avoid potential confusion
NAME          IMAGE                                COMMAND                                SERVICE    CREATED        STATUS          PORTS
fakenet-ng-server integrated-malware-lab-fakenet-ng  "/entrypoint.sh" fakenet-ng  14 seconds ago Up 7 seconds (healthy) 0.0.0.0:80->80/tcp, [::]:80->80/tcp, 0.0.0.0:443->443/tcp, [::]:443->443/tcp, 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp, 0.0.0.0:8081->8081->8080/tcp, 0.0.0.0:9081->8080/tcp, [::]:9081->8080/tcp, [::]:9081->8080/tcp, 0.0.0.0:8444->8443/tcp, 0.0.0.0:9444->8443/tcp, [::]:8444->8443/tcp, [::]:9444->8443/tcp, 0.0.0.0:9900->9000/tcp, [::]:9900->9000/tcp
inetsim-server integrated-malware-lab-inetsim      "/entrypoint.sh" inetsim     10 seconds ago Up 6 seconds (healthy) 0.0.0.0:21->21/tcp, [::]:21->21/tcp, 0.0.0.0:25->25/tcp, [::]:25->25/tcp, 0.0.0.0:53->53/tcp, [::]:53->53/tcp, 0.0.0.0:53->53/udp, [::]:53->53/udp, 0.0.0.0:8082->80/tcp, 0.0.0.0:9080->80/tcp, [::]:8082->80/tcp, [::]:9080->80/tcp, 0.0.0.0:8445->443/tcp, 0.0.0.0:9443->443/tcp, [::]:8445->443/tcp, [::]:9443->443/tcp

Next Steps:
1. Test DNS:      nslookup example.com
2. View logs:     docker-compose logs -f
3. Run test:      bash scripts/test-connectivity.sh
4. Full guide:    cat docs/SETUP_GUIDE.md

Useful Commands:
Stop services:    docker-compose stop
Restart:          docker-compose restart
View logs:        docker-compose logs -f inetsim
Clean up:         sudo bash scripts/cleanup-routing.sh && docker-compose down

▲ Important:
- Lab network is isolated: 172.20.0.0/24
- INetSim DNS:           172.20.0.2:53
- FakeNet-NG HTTP:       172.20.0.3:80/443
- Logs stored:           shared/logs/

[✓] Integrated Malware Analysis Lab is READY!
```

FULL MODE

Toàn bộ traffic bị chặn và redirect tới INetSim. Không có kết nối internet thật – môi trường cách ly hoàn toàn.

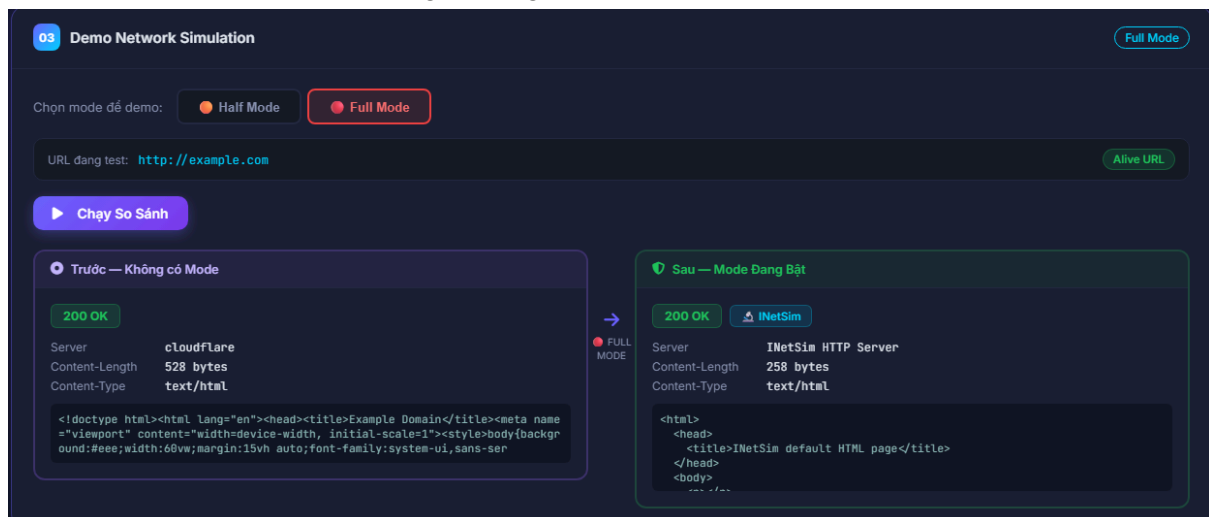
DNS: INetSim DNS 172.20.0.2:53

HTTP: INetSim HTTP 172.20.0.2:80

Use case: Phân tích malware chưa biết nguồn gốc, môi trường sandbox hoàn toàn cách ly.

Flow:

- + Package gửi request tới domain bất kỳ
- + DNS bị chặn – toàn bộ resolve qua INetSim DNS
- + INetSim trả về IP giả 127.0.0.1 cho mọi domain
- + HTTP request bị redirect tới INetSim HTTP server
- + INetSim trả về response giả lập, ghi lại toàn bộ hành vi



HALF MODE

Chặn và giả lập các URL đã chết (dead URLs). Các domain còn alive được kết nối bình thường.

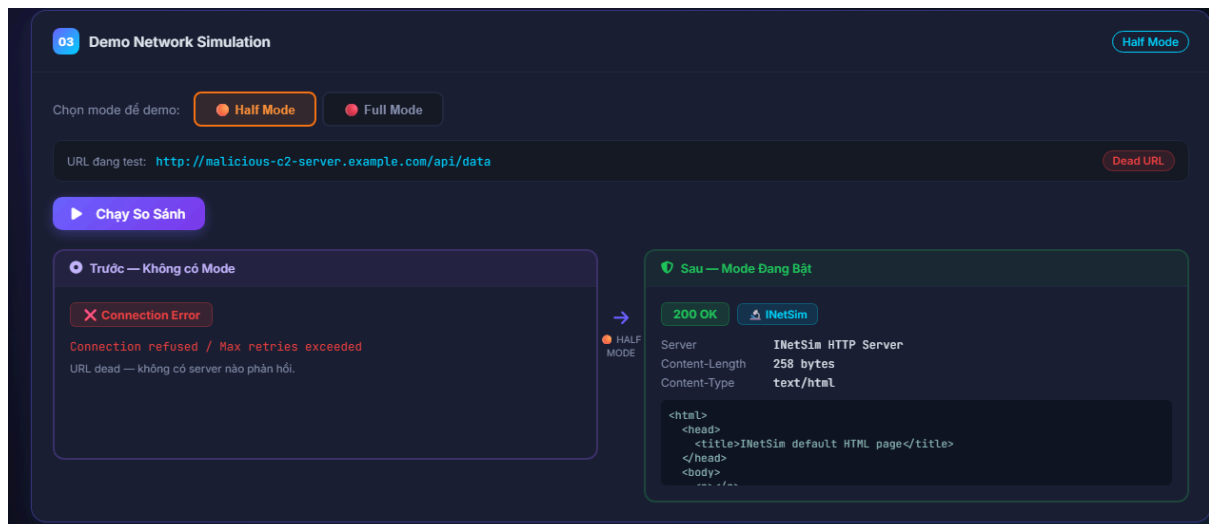
DNS: Conditional: dead → INetSim DNS / alive → System DNS

HTTP: Dead URL → INetSim HTTP / Alive URL → Direct

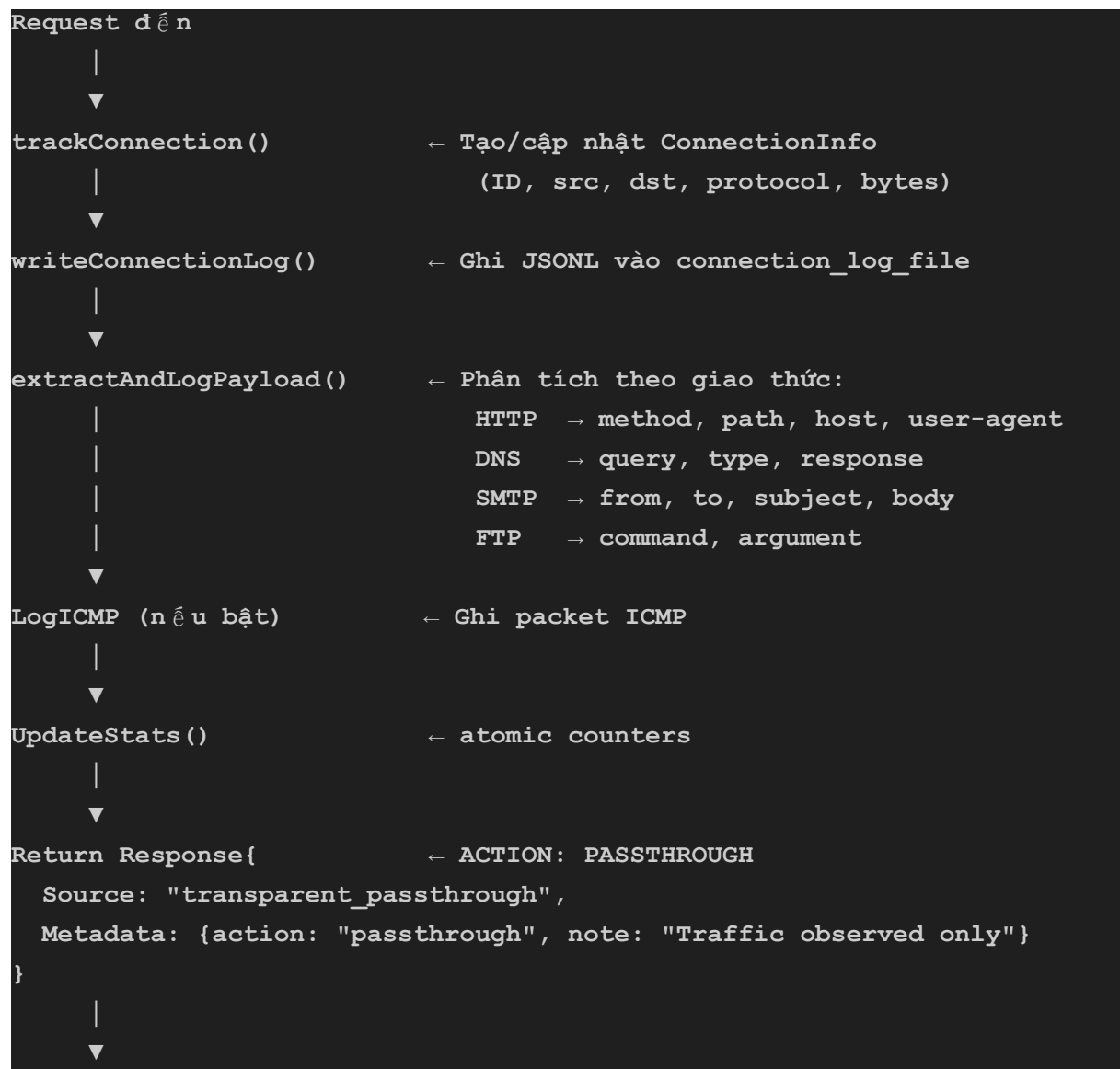
Use case: Phát hiện malware dùng C2 server đã dead – redirect để giả lập response thay vì để fail.

Flow:

- + Package gửi request tới một URL
- + Hệ thống kiểm tra URL còn alive không
- +  URL alive → cho qua kết nối internet trực tiếp
- +  URL dead → redirect DNS & HTTP tới INetSim
- + INetSim giả lập response, ghi lại hành vi của dead URL



Transparent mode



Traffic đến destination ← KHÔNG BỊ THAY ĐỔI

Transparent mode không thay đổi traffic, chỉ quan sát. Lấy nguyên tắc từ siemens/sparring “will not alter any transmitted data”. Tất cả các trường phản ánh chính xác hành vi của transparent.go - handler chỉ ghi log, không intercept hay block.

The screenshot displays the 'Demo Network Simulation' interface. At the top, it shows '03 Demo Network Simulation' and a 'Transparent Mode' button. Below this, there are three mode selection buttons: 'Half Mode' (orange), 'Full Mode' (red), and 'Transparent' (green, which is selected). A text field shows the 'URL đang test: http://c2-tracker.evill.example.com/beacon' with an 'Observe Only' button to its right. A 'Chạy So Sánh' button is also present. The main area is divided into two panels. The left panel, titled 'Trước — Không có Mode', shows a 'Connection Error' message: 'Connection refused - URL dead/unreachable (no server responding)' and 'URL dead — không có server nào phản hồi.' The right panel, titled 'Sau — Transparent (Observe Only)', shows 'CONNECTION INFO' with details: Source 192.168.1.50:55001, Destination 203.0.113.42:80, Protocol HTTP, and Bytes 1,432 bytes. It also shows 'EXTRACTED PAYLOAD' with a GET request to /beacon, Host: undefined, and User-Agent: Python-urllib/3.9 (suspicious). A green arrow labeled 'TRANS PARENT' points from the left panel to the right. At the bottom of the right panel, it states 'Traffic NOT modified — observation only'.

Transparent mode before = fail, after = success vì không có sever giả lập traffic chỉ được ghi nhận và truyền qua nguyên vẹn. Sever thật vẫn không tồn tại nên không có body trả về, nhưng obsever đã capture được toàn bộ metadata và payload của request.

**Full Mode**

Cách ly hoàn toàn

**Half Mode**

Chặn dead URLs

**Transparent**

Quan sát thụ động

**Transparent Mode****Passive Safety**

Không sửa đổi bất kỳ traffic nào. Chỉ quan sát thụ động, theo dõi kết nối và trích xuất payload từ các giao thức được hỗ trợ. Lấy cảm hứng từ siemens/sparring.



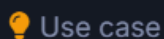
DNS

Pass-through – DNS không bị chặn hay redirect



HTTP

Pass-through – HTTP/HTTPS không bị sửa đổi



Use case

Phân tích pháp y thụ động (passive forensic), baseline profiling, phát hiện C2 channels mà không làm thay đổi hành vi malware.

Flow:

- Package gửi request tới bất kỳ đâu
- Transparent handler ghi nhận connection info (src, dst, protocol)
- Nhận dạng giao thức qua port mapping và payload inspection
- Trích xuất dữ liệu từ HTTP, DNS, SMTP, FTP (không lấy password)
- Traffic được pass-through KHÔNG BỊ SỬA ĐỔI
- Toàn bộ kết nối và payload ghi vào JSONL log file

**Mở Transparent Observer ↓**